



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Multiuser Conversational Agents Based on Large Language Models

TESI DI LAUREA MAGISTRALE IN

COMPUTER SCIENCE ENGINEERING - INGEGNERIA INFORMATICA

Francesco Dubini, 10617059

Advisor:

Prof. Franca Garzotto

Co-advisors:

Prof. Enrico Costanza

M.Sc. Mathyas Giudici

Academic year:

2023-2024

Abstract: The integration of Large Language Models (LLMs) into multi-user conversations has the potential to influence social dynamics and decision-making processes. This Thesis explores the role of an Artificial Intelligence (AI) powered chatbot as a moderator in group discussions, investigating whether it can replicate human moderation functions by guiding conversations toward more effective outcomes. The research employs an online study where participants engage in a murder mystery task, designed to stimulate collaboration and decision-making, under two conditions: the control condition (without the LLM moderator), and the experimental condition (with the LLM-powered moderator).

Our findings indicate that while LLM-powered moderation can subtly influence participation dynamics and focus discussions, its overall impact on decision accuracy was limited. Pilots suggested that some limitations may be attributable to characteristics of the participant pool, which could have influenced the results. Nevertheless, we report all analyses, design steps, and adaptations in detail to support further research in this field. These insights provide a strong foundation for future work on enhancing AI-driven moderation in collaborative settings.

Key-words: Human-Computer Interaction, Human-AI Interaction, LLM, Moderation, Multi-users Chatbot

1. Introduction

In recent years, the transformer architecture [37] has sparked a revolution in artificial intelligence, leading to significant advancements in natural language processing and understanding. One of the most impactful outcomes of this evolution is the development of Large Language Models (LLMs). These models have demonstrated an extraordinary ability to generate human-like text, making them valuable tools in a variety of applications, particularly in the realm of conversational agents. Traditionally, LLMs have been employed in one-on-one chatbot interactions, where they respond to individual user prompts. However, this Thesis explores a more complex and less-explored scenario: the integration of an LLM-powered chatbot as an artificial moderator in multi-user conversations.

This shift in focus leads to the central research question of this Thesis:

How do social dynamics and decision-making processes change with the introduction of an artificial moderator in a conversation?

Human moderators play a crucial role in discussions, guiding the flow of conversation, ensuring equitable participation, and managing conflicts. Our aim is to investigate whether an LLM moderator can replicate some of these functions by influencing the balance of participation, the flow of discussion, and the efficacy of decision-making in group settings. This research is motivated by the limited existing literature on multi-user generative chatbots, where the most significant contributions have emerged within the last year [26]. Ultimately, this Thesis does not focus on building the "perfect" LLM moderator but on gathering direct data from human-AI interaction that could inform future advancements in this field. By employing a simple, baseline model of an artificial moderator, we aim to uncover insights that might guide the design of more sophisticated systems in the future.

To explore our research question, we conducted an online study using participants recruited from Prolific, an online platform that facilitates academic research by connecting researchers with diverse participants. Our study is based on a murder mystery game inspired by a previous investigation by Stasser and Stewart [34], which aimed to explore whether the nature of the decision task influenced its outcome. During the study, three participants are each given a unique, critical piece of the same puzzle. Only through a thorough discussion among them can they uncover the real culprit. In contrast, our focus is on using the mystery to induce discussion, a concept previously implemented in a study on user balance in multi-user conversations [4].

In the original mystery setup, three participants were each provided with different transcripts about a fictional case and given an hour to review their materials. They then had an additional hour to discuss the case and determine who the culprit was. This discussion phase was critical, as each participant held unique pieces of information, necessitating collaborative dialogue to identify the true culprit.

In this Thesis, we have digitized the original study by transforming the talking stage into an online chat format, incorporating an integrated vote button to conclude the discussion. To explore our research question, we implemented an LLM-powered moderator within the chat to observe its impact on participants and the final voting outcome. The study was conducted under two conditions: a control condition without moderation, to assess the performance of the online version of the study, and an experimental condition with the LLM moderator present in the chat.

By running the study 20 times (10 in each condition), we measured the success rate of groups in identifying the culprit, alongside other metrics adapted from the original study. This Thesis will provide a detailed account of the methodology, the technical solutions employed to digitize the study, and the results of our analysis. The insights gathered from this pilot study offer a foundation for understanding how AI moderation might influence multi-user conversations and decision-making processes.

The Thesis is structured as follows. Section 2 covers the current state-of-the-art regarding AI in the field of moderation. Section 3 explains how the original study operated and how we adapted it to an online setting. Section 4 highlights the technical solutions behind our version of the study, together with an explanation of how the final prompt was obtained. Section 5 presents the data collected and its analysis, Section 6 reports the limitations we observed in our study. Lastly, Section 7 presents the conclusions with future directions for research work.

2. Related Work

In recent years, the emergence of the transformer architecture [37] has profoundly impacted artificial intelligence (AI), demonstrating high performance across various fields [14, 21]. Notably, the GPT architecture [30] has propelled advancements in text generation at an unprecedented pace, culminating in the development of the GPT-3 model [6], widely known as ChatGPT. Following GPT, many other Large Language Models (LLMs) were developed, including examples such as Claude [2], Gemini [18], and LLaMA [35]. LLMs have proven useful in many fields, including multimodal image classification [7], summarization [10], and prediction [36].

With the development of these tools, the core concept of chatbots has significantly evolved. The formal definition of a chatbot is: *"A chatbot can be defined as a computer program designed to interact with users using natural language or text in such a way that the user believes they are engaging in a dialogue with a human"* [20]. However, the term chatbot is now often associated with LLMs.

Chatbots can be categorized into three types: rule-based, generative, and hybrid [16], with LLMs representing just one of these categories. Rule-based chatbots function by scanning user utterances and responding based on a predefined set of rules. The first chatbot, ELIZA [40], operated under this paradigm—it rudimentarily parsed user input for patterns and generated responses using pre-programmed templates. Rule-based chatbots continue to play a significant role in domains such as customer support [42] and social media [24], largely facilitated by tools like Google’s Dialogflow [31].

The second category of chatbots is generative chatbots. These chatbots generate replies using machine-learning techniques based on the context of the conversation. This allows them to produce dynamic responses, as they are not constrained by predefined rules. The key evolution over the years has been in the machine learning techniques used for text generation [1]. What began with Markov Chain-based text prediction has evolved into modern LLMs and the use of fine-tuning, Retrieval-Augmented Generation (RAG), and prompt engineering.

Prompt engineering is an emerging technique that leverages the potent capabilities of LLMs for optimizing and guiding their output across a wide range of applications [9]. Recent studies have demonstrated that a well-engineered prompt can yield performances comparable to that of a fine-tuned model [6], making AI accessible to an increasing number of researchers and fields, with some disciplines even developing their own specialized techniques [5]. However, due to the low interpretability of LLMs, there is no rigorous, standardized process for prompt engineering; much of it relies on the intuition of developers, who must often refer to existing literature for effective prompting norms [3]. Some research is being conducted to automate the exploration of prompt spaces [41], but much of the work remains task-specific.

The third and final category encompasses hybrid approaches, which may utilize a rule-based framework with LLM generation as a fallback [16]. This category remains relatively underexplored, and existing research on it is limited. The three approaches explained before are typically based on a dyadic setting, where chatbots are designed for one-on-one interactions with users. As a result, the multi-user framework, according to our research, remains relatively unexplored [43], with many issues still lacking clear solutions, and the most relevant work being published very recently [26].

Since our focus is on introducing an LLM chatbot in a multi-user setting, we reviewed the literature for suitable tasks it could perform. Kim et al. [22] used a rule-based chatbot to facilitate discussions around the trolley problem, while Microsoft’s Calendar Help [12] assists users in scheduling meetings through a chatbot agent. Previous research has demonstrated the potential of conversational agents to facilitate discussions without relying on LLMs [12, 22, 32]. Building on this, we decided to explore moderation aided by an LLM.

To better understand the landscape of moderation, we also explored related research in other disciplines, such as political debates [38], moderation in learning facilitation [19], and forum discussions [28]. More generally, Kruger’s work on focus groups [23] provides a useful starting point by outlining potential behaviors that a moderator should exhibit. However, as we later discovered during development, these guidelines were too simplistic and lacked the depth needed to fully capture the complexity of a moderator’s behavior. Due to this, and the diversity of tasks involved, it becomes clear that there is no definition of a "universal" moderator applicable across all contexts. Each task, to some extent, requires its own set of rules and criteria for moderation.

By analyzing literature on AI-driven moderators, we found that Wagner’s study [39] proposed various moderation strategies, offering a promising approach; however, no statistically significant differences between these strategies were found. We also reviewed the work of Chiang et al. [11], in which the moderator constantly challenged participants’ input to stimulate deeper discussion. While innovative, we concluded that this style of

moderation was unsuitable for our task, as we hypothesized that challenging three participants simultaneously would create excessive confusion and disrupt the flow of conversation.

A key focus we found in a multi-user setting is turn-taking; determining when the bot should intervene in a conversation. Based on the reviewed literature, we identified three main solutions to this problem: *direct call*, *timer*, and *rule-based*.

Direct call refers to situations where a multi-user chatbot intervenes when explicitly addressed. A clear example is UnevenChef [17], a study that investigates uneven treatment by an AI, named Emma, within a group. In this study, users were placed in a common chat and could invoke the bot only by directly addressing it (e.g., @Emma).

Timer represents cases where the intervention of the bot in the group is dictated by a timer of some sort. An example of this behavior is found in IntroBot by Shin et al. [32], where the moderator is scheduled to intervene at specific points during the conversation.

Lastly, the most common approach is the *rule-based* method, in which the chatbot takes action based on pre-defined rules. These rules can vary; some may involve parsing user content and following a flowchart [22], while others may incorporate more complex machine learning techniques [39].

These three techniques are not mutually exclusive. For example, IntroBot [32] leverages both *timer* and *rule-based* approaches to build an online discussion facilitator that utilizes users' past social media data. Another instance of multiple techniques being employed simultaneously is Microsoft's MUCA [26], a framework designed for multi-user LLMs. MUCA implements all of the aforementioned techniques together; an LLM is provided with the state of the chat and selects from a set of predefined actions (or rules) to execute. This can be triggered in several ways: after a certain number of messages, at specific time intervals, or when the bot is directly addressed. However, to the best of our knowledge, there is no instance in which an LLM is given full autonomy in deciding both what and when to intervene. This thesis aims to explore the possibilities of such a baseline.

When evaluating the moderator, our review of the literature revealed a lack of standardized metrics for assessing performance. Most studies rely on subjective measures, such as user satisfaction at the experiment's conclusion. While many of the studies we reviewed focus on qualitative analysis [11, 26, 32], our work aims to be grounded in quantitative methods by leveraging the discussion metrics used in Stasser's original study [34]. As far as we know, no other research utilizes pre-established metrics in the same way that ours does.

3. Method

Our primary objective was to understand how social dynamics and decision-making processes change when an LLM-powered moderator is introduced. To achieve this, we aimed to evaluate the capabilities of an off-the-shelf LLM, without any fine-tuning or prompting for a specific type of conversation. To conduct this evaluation, we needed a suitable task that would not only induce discussion but also provide clear metrics to measure the quality of the conversation. We selected Stasser and Stewart's protocol [34], as its murder mystery task is a well-established example in the literature of a setting that stimulates group discussion, while also providing a decision-making task with appropriate metrics to evaluate different instances. This same protocol was similarly applied by Bachour et al. [4], in a previous study on discussion balance in a group.

3.1. Stasser's Reference Study

In his study, Stasser sought to determine whether the nature of a task influences its outcome. Specifically, he hypothesized that if a decision-making task implies the existence of a correct answer, participants would approach it differently.

3.1.1 Research Design

The original study work recruited a total of 467 students, all of which had to complete it as a requirement for a psychology course. The study was run in a $2 \times 2 \times 2$ factorial design, obtained by varying **group size**, **task set**, and **information distribution**.

The **group size** was varied between 3 and 6-person groups: the students were split into 3-person ($N=43$) groups and 6-person groups ($N=44$).

Since the goal was finding out whether the nature of the task influences the outcome, Stasser and Stewart ideated two different task modalities:

- *Solve*: Given the mystery, the participants are told that only one culprit could have committed the crime. They need to pinpoint a single culprit among the three suspects.
- *Judge*: Given the mystery, the participants are told the data is not sufficient to pinpoint a single culprit. They are needed to make a ranking from most guilty to guilty.

The paper argued that, since the *Judge* set implies the data is insufficient, the information search during discussion would be less thorough. This occurs because the group tends to settle once an agreement is reached, without further exploring unshared information.

The **task set** was thus varied between *Solve* and *Judge* task set conditions as part of the paper’s research question.

The last condition, varied the **information distribution** between *Hidden Profile* and *All Shared*. The concept of a *Hidden Profile* was previously introduced by Stasser himself [33]; it is described as a pattern of unshared information in which « *a superior decision alternative exists but its superiority is hidden from individual group members because they each have only a portion of information that supports this superior alternative* ». Within the study’s context, the participants in the *Hidden Profile* condition have transcripts with a select number of clues defined as **critical** that are uniquely distributed within their group, while participants in the *All Shared* condition had no clues hidden from them. Given the importance of the *Hidden Profile* to both the original and our study, it is essential to explain in detail how the clues are divided among transcripts in different groups. Let’s briefly review how all of the 56 pieces of information are divided in the murder mystery to better understand the *Hidden Profile*. The information is classified into 3 categories: **details**, **noncritical**, and **critical**.

There are 32 pieces of information classified as **details**, these are facts or background information that neither incriminate nor exonerate any of the three suspects: Mickey Malone, Billy Prentice, and Eddie Sullivan (the culprit).

The remaining 24 pieces of information are defined as **clues**, and they either incriminate or exonerate a specific suspect. 15 out of these 25 clues are defined as **noncritical**, and they implicate the wrong suspects (Billy and Mickey). These noncritical clues are shared among all versions.

The murder mystery implements the *Hidden Profile* through the last 9 clues, defined as **critical**, which play a key role in incriminating the real culprit (Eddie) and exonerating the other two suspects. Three of these nine clues exonerate Mickey, three exonerate Billy, and the final three incriminate Eddie. A *Hidden Profile* version contains only three critical clues: one that incriminates Eddie, one that exonerates Billy, and one that exonerates Mickey. A summary of the information distribution across the study settings is illustrated in Figure 1.

In summary, each 3-person group receives three different versions of the information, with each version containing 32 details and 18 clues—15 of which are noncritical and 3 of which are unique critical clues. For the 6-person groups, the first three participants received the same versions as those in the 3-person groups, while the other three participants received versions containing 32 details and 15 noncritical clues, with no critical clues included.

3.1.2 Procedure

During the original study, participants were met together in a room in which they were told the study was about *"how groups use information to make decisions"*. They were then all told that: *"The detectives in the case do not have sufficient evidence to charge anybody for the homicide..."*, then depending on the condition:

- *Judge*: *"... we are asking you to use the available evidence and decide which of the suspects seems most likely to have committed the crime."*
- *Solve*: *"... only one of the suspects could have committed the crime. We would like you to read over the materials carefully and to decide who you think committed the crime."*

Participants then had 30 minutes to read all of the materials, consisting of a 27-page booklet that included the interviews with other supporting materials such as a map, a handwritten note, and a newspaper article.

Afterward, each participant completed a private questionnaire specifying who, based on the transcript they had just read, was the culprit (or provided a ranking of culprits in the *Judge* condition).

After the questionnaire was done, groups met in different rooms to discuss the suspects. They had a maximum of 30 minutes to reach a consensus on the culprit (or a ranking of culprits, in the *Judge* condition). If a group reached a solution before the 30-minute mark, they were free to leave early. All discussions were recorded for

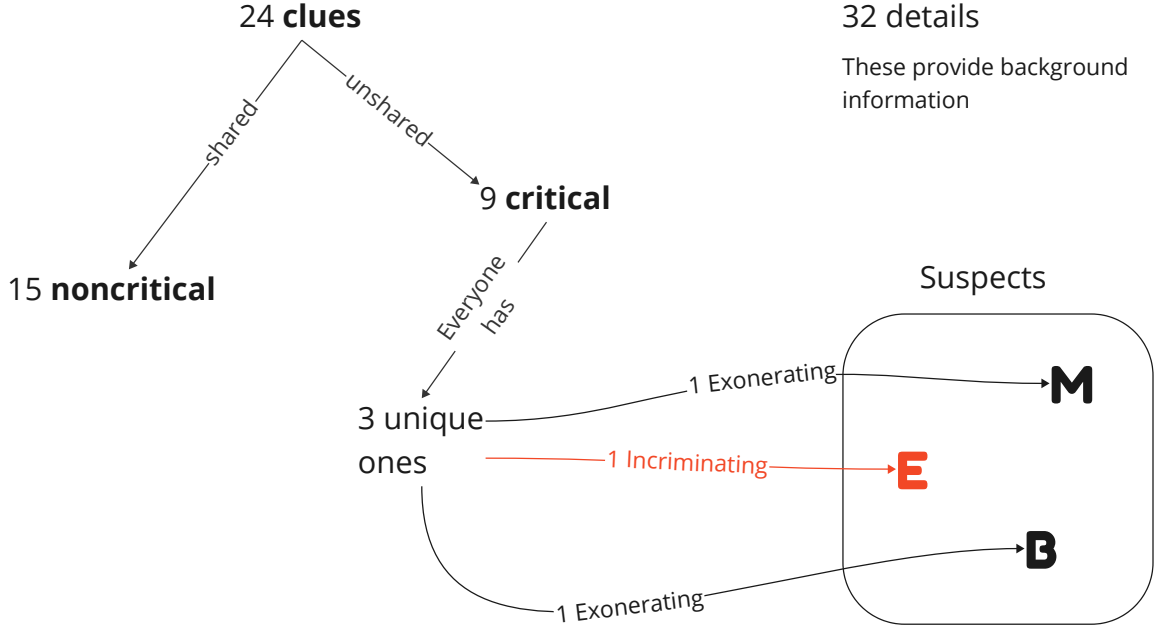


Figure 1: How the information is shared within the *Hidden Profile*, and how the critical clues incriminate or exonerate **E** (Eddie), **B** (Billy), and **M** (Mickey).

further analysis. After the discussion ended, participants rated the sufficiency of information for determining who was the guilty suspect.

3.2. Our Study

3.2.1 Research Design

Out of all the conditions used in the original protocol, we opted to just use the 3-person group, *Solve* and *Hidden Profile* conditions. We decided not to opt for the 6-people groups since we were not able to forecast the LLM generation capabilities with such a number of messages involved (e.g., Prompt Leaks [25]), and it would end up being a costly unknown experiment. Additionally, having any number of participants within a group other than the one proposed in the original paper would require us to change the original transcripts or settings (something that, for example, Bachour et al.[4] did by transforming the mystery into a 4-person group task), losing the possibility of making any comparison between our results with the original ones. We further chose the *Solve* condition because we were not interested in the difference a *Judge* setting would make, and as stated in the original paper, the *Solve* condition usually leads to more thorough discussions. We lastly didn't explore the *All Shared* condition as, according to the original paper, having all the information available usually easily leads to the solution, resulting in short and trivial discussion.

To investigate our research question and the efficacy of our LLM-powered moderator, we introduce both a control and an experimental condition. The control condition, without the moderator, serves as a baseline for comparison with the original study. The experimental condition, by contrast, includes the LLM-powered moderator in the chat to evaluate its effects.

3.2.2 Procedure

We will now explain how we adapted the procedure of the original study within our chosen conditions (3-person, *Solve*, and *Hidden Profile*) into an online setting. A visual representation of the adaptation is depicted in Figure 2.

Each Stage involves the users receiving specific information and performing actions on an online Page. The first Page called the *Information Page* (Figure 3a), informs the participants of the study's purpose:

"The purpose of this study is to observe how a group of participants discusses via an online chat to reach

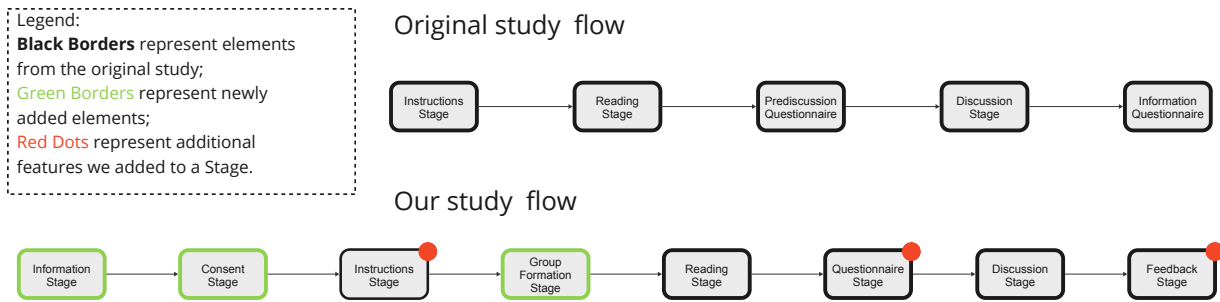


Figure 2: Difference between the original study stages and our adaptation

a decision about a fictional murder mystery, and how interactive chat technology might influence this discussion."

Additionally, it is asked for participants not to leave during the experiment. This was one of the main challenges when deploying the study online via Prolific. Since we cannot ensure that all participants will complete the study from start to finish—and due to Prolific’s policies, we couldn’t require them to do so—we had to rely on the participants’ willingness to stay engaged. If a participant were to leave, for example, during the discussion, it would invalidate the entire group, leading to a significant loss of money as we would still need to compensate the remaining two participants without receiving any usable data.

We also clarify how the data will be handled, provide contact information for any complaints and explain the process for withdrawing from the study.

The following Page is the *Consent Page* (Figure 3b), used to gather agreement on the basic terms and conditions regarding the study.

Participant Information

Murder Mystery Study

The purpose of this study is to observe how a group of participants discusses via an online chat to reach a decision about a fictional murder mystery, and how interactive chat technology might influence this discussion.

The total duration of the study is of about 1 hour. If you agree to participate, we kindly ask that you make your best effort to complete the study from beginning to end. This is because if you stop the study in the middle, two other participants will also have to stop the study. On this note, please keep the browser window (or tab) of the study always open and visible, otherwise, you will appear as disconnected from the study and we will ask you to return it. We furthermore kindly ask you to disable any Adblock while doing the study, as they could cause some issues.

If you agree to participate, you will be asked to read a murder mystery fictional short story (for about 30 minutes) and then discuss (for about 30 more minutes, via text chat) with two other participants to collaboratively identify the culprit of the fictional crime. You will also be asked to complete one brief questionnaire after you read the short story and another one after the discussion, at the end of the study.

There are no particular risks associated with your participation other than those associated with the use of standard computer equipment, and reading crime fiction.

Data and Information.

All data will be handled according to the GDPR and will be kept anonymous. Only UCL researchers working with Dr. Enrico Costanza will analyse these data.

Any information that is obtained in connection with this study and that can be identified with you will remain confidential and will be disclosed only with your permission or as required by law. The information collected during the experiment will be kept separately from your personal identity. The information will be collected and stored in a password-protected computer.

Withdrawing from the Study.

If you decide you do not want the data to be recorded, or if you decide at a later date that you do not want these data to be kept in the form specified above, you may request that we delete the data from our dataset by contacting the principal investigator (e-mail: e.costanza@ucl.ac.uk), within the next 20 days. After the next 20 days, any data will be fully anonymized (i.e. anything that may identify the data as yours will be removed) and may be made available to other researchers – as this can be helpful for other researchers working on similar topics.

You may withdraw at any time and for any reason during the study. You may access or withdraw your data at any time and for any reason within the next 20 days after the study ends when all the data is being anonymised or securely destroyed.

Concerns or Complaints.

Should you have any concern or complaint, you can contact us at any point via Prolific messages or via email (E. Costanza e.costanza@ucl.ac.uk).

It is up to you to decide whether or not to take part. If you decide to take part you will be given this information sheet to keep and be asked to sign a consent form. If you decide to take part you are still free to withdraw at any time and without giving a reason.

[Continue](#)

Informed Consent Form

Project Title: Murder Mystery Study

Project ID No: UCLIC_2022_004_costanza

This study has been approved by the UCL Interaction Centre (UCLIC) Research Department's Ethics Committee

Contact Details of Investigators.

Principal investigator:
 Dr. Enrico Costanza
 UCL Gower Street
 London WC1E 6BT
 United Kingdom
 +44 (0)20 7679 718
 email: e.costanza@ucl.ac.uk

Name and contact details of the UCL Data Protection Officer:
 Alexandra Potts, data-protection@ucl.ac.uk

Participant's Statement.

If you have any questions arising from the Information Sheet or explanation already given to you, please ask the researcher before you decide whether to join in.

I, the participant, confirm I understand that by agreeing to the items below I am consenting to participating in this study. I understand that it will be assumed that not checking the box mean that I DO NOT consent to that part of the study. I understand that by not giving consent for any one element I may be deemed ineligible for the study.

- I confirm that I have read and understood the Information Sheet for the above study. I have had an opportunity to consider the information and what will be expected of me. I have also had the opportunity to ask questions which have been answered to my satisfaction and would like to take part in the survey study.
- I consent to participate in the study. I understand that my personal information, in the form of this consent form, will be used for the purposes explained to me. I understand that according to data protection legislation, "public task" will be the lawful basis for processing.
- I understand that all personal information will remain confidential and that all efforts will be made to ensure I cannot be identified.
- I understand that my data gathered in this study will be stored anonymously and securely. It will not be possible to identify me in any publications.
- I understand that my information may be subject to review by responsible individuals from the University for monitoring and audit purposes.
- I understand the potential risks of participating and the support that will be available to me should I become distressed during the course of the research.
- I understand the direct/indirect benefits of participating.
- I agree that my anonymized research data may be used by others for future research. (No one will be able to identify you when this data is shared).
- I am aware of who I should contact if I wish to lodge a complaint.
- I agree for the data specified in the information sheet to be collected and stored on a secure server, and for an aggregate or anonymous version of this data to be made publicly available.

☒ I consent and agree to the terms (items 1 – 10) above.

[Continue](#)

(a) Information Page.

(b) Consent Page.

Figure 3: Snapshots of Pages one and two of the study.

The *Instructions Page* (Figure 4a) details each stage of the study and how long it takes. It also includes a comprehension check, used to filter out participants who fail to demonstrate a basic understanding of the

study's rules and structure. In compliance with Prolific's policies, each participant has two chances to correctly complete the check before being "rejected" by the system, meaning they won't be able to participate in the study again.

The *Group Formation Page* (Figure 4b) functions as a loading screen, asking users to wait for a group to be created. Within our design, we decided to dynamically assign incoming participants to groups made of three versions (A,B, and C) that make up the *Hidden Profile*. A further breakdown of the algorithms involved will be later explained at Section 4.1.1.

Instructions

Murder Mystery Study

The purpose of this study is to observe how a group of participants discuss via an online chat to reach a decision about a fictional murder mystery, and how interactive chat technology might influence this discussion. If you agree to participate, you will be asked to read some background material about a murder mystery fiction (a short story) and then discuss (via text chat) with two other participants to find the culprit in a cooperative way.

The total duration of the study is of about 1 hour. If you agree to participate, we kindly ask that you make your best effort to complete the study from beginning to end. This is because if you stop the study in the middle, two other participants will also have to stop the study.

The study includes five stages:

- 1. Group formation stage**
During this stage, we kindly ask you to wait for 2 other participants to join the study. It should normally only take few seconds, and hopefully in the worst case no more than one or two minutes. Once the group is made, it will remain as such until the end of the 1 hour study. So, we kindly ask that you make your best effort to complete the study from beginning to end. This is because if you stop the study in the middle, two other participants will also have to stop the study.
- 2. Reading stage**
After a group is formed, you and the other two participants will move to the reading stage: you will have 30 minutes to read and understand a short story about a fictional murder mystery. The text is mostly transcripts of fictional police interviews. Beware, the story documents will not be available later on during the study: we suggest you take all the 30 minutes you have to read them carefully so you understand and remember as much as possible. Please keep the story information strictly confidential (do not share it in any form) as it would invalidate our study for further participants.
- 3. Questionnaire stage**
After reading, you will be asked to fill out a brief questionnaire containing 7 multiple-choice questions related to the text. This questionnaire should take up to 2 minutes to complete and will assist us in evaluating the clarity of the reading material.
- 4. Discussion Stage**
In the discussion stage, you and the other two participants will have 30 minutes to discuss the material you read and decide who you think the culprit of the murder mystery is. Each participant will be asked to vote until you reach consensus. We advise you to discuss thoroughly before voting, because only the co-operation of all three participants allows for the successful resolution of the mystery. Please, avoid random guessing or writing few messages as they would ruin your contribution to the study.
- 5. Feedback**
At the end of the study a brief questionnaire will ask for your general feedback about the study. This should not take more than 2 minutes to complete.

Multiple choice questions:

The reading and discussion stage will last:

- ☐ 30 minutes both
- ☐ 1 hour both
- ☐ 15 minutes and 45 minutes respectively

In the study there are/is:

- ☐ Just me
- ☐ Two participants
- ☐ Three participants
- ☐ Four participants

The information in the reading stage:

- ☐ Will be accessible at any point of the discussion stage
- ☐ Is not that necessary for the discussion stage
- ☐ Will be crucial for the discussion stage

What does the study revolve around?

- ☐ Murder Mystery
- ☐ GDPR Laws
- ☐ Time management

(a) Information Page.

Waiting for other users to join
2/3 users have joined



(b) Group Formation Page.

Figure 4: Snapshots of Pages three and four of the study.

The *Reading Page* (Figure 5a) is where participants can read the digitized version of all the materials available from the original study. The interviews, along with additional materials, are placed in a JavaScript iframe that can be scrolled vertically. We ensured that the text could not be highlighted to prevent it from being copied and pasted into language models, or saved for further use. Each participant will contain a different version to recreate the original *Hidden Profile*. The role matching the version (A, B or C) is assigned during the *Group Formation Page* and stays the same throughout the rest of the study.

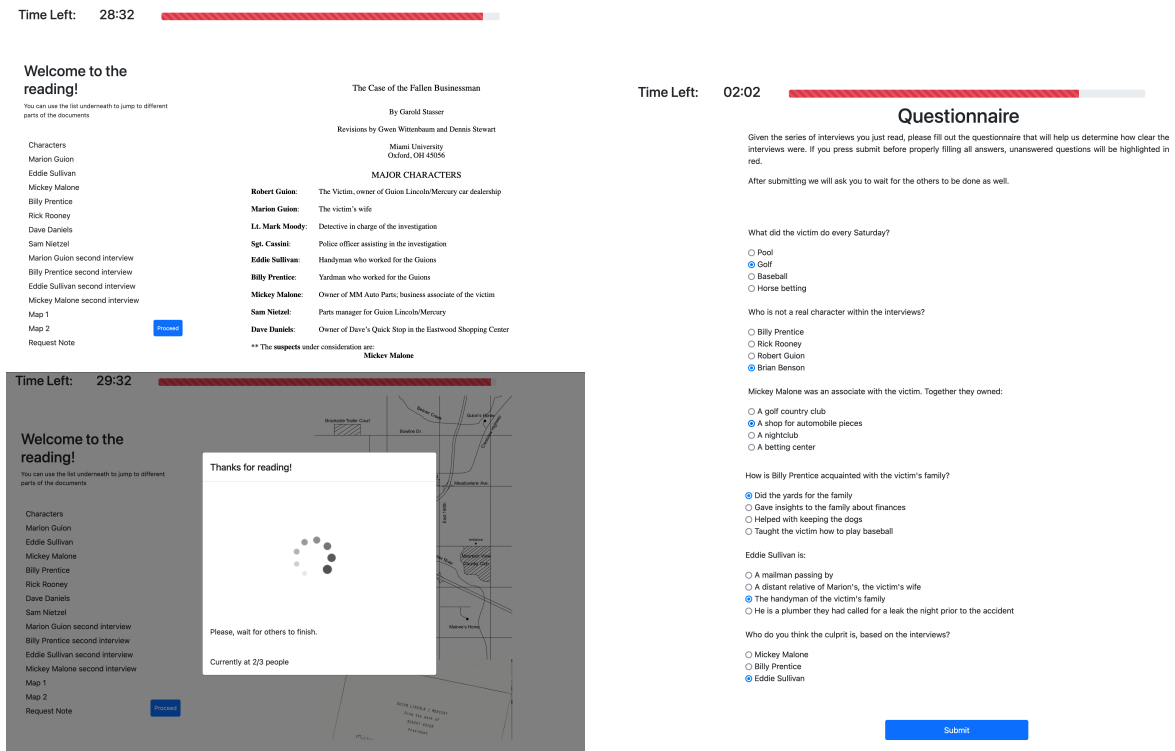
On the left side of the Page is a selector that allows participants to navigate through the various parts of the interviews for easier access to the documents. Next to the selector is a "Proceed" button that, when clicked, prompts the user to confirm if they are truly finished, as they will not be able to reaccess the documents. If the user confirms, a waiting pop-up asks them to wait for the other two participants, displaying how many have completed the reading so far (Figure 5a).

At the top of the Page, there is a red scrollbar that visually represents the remaining time for reading. Additionally, the exact number of minutes left is displayed. If the time expires, a pop-up prompts the user to confirm that the time is up. Once confirmed, the user is automatically directed to the 'Waiting for others' pop-up.

We decided against setting a fixed reading time for all participants, as we feared participants who finished early would leave the study if they were too impatient to wait for others. Moreover, we were unsure how the original half-hour would translate to an online format. Allowing users to finish early, if the time is sufficient, helps us evaluate the digitization of the study by providing precise data on how much time each participant spends reading. A side effect of this approach is the ability to identify "unmotivated" participants; if a participant spends significantly less time on the documents compared to the mean, it is likely that they are not fully engaged, which could lead to less reliable results.

The *Questionnaire Page* (Figure 5b) was designed to gather participants' opinions on the culprit before the discussion, similar to the original study. We also added some trivial questions intended to serve as an "extra attention check". Although we could not exclude participants who failed these basic questions regarding the transcripts, these questions helped us distinguish between participants who paid enough attention and those who did not.

As the questionnaire is relatively short and easy, we give each user two minutes and thirty seconds to complete it, the timer and the scrollbar on the top of the Page portray the time. If the time runs out, the questionnaire is submitted incomplete and users are pushed to the following Page.



(a) Reading Page (above), Done reading (below).

(b) Questionnaire Page.

Figure 5: Snapshots of Pages five and six of the study.

The *Discussion Page* (Figure 6a) is where the participants can interact with each other and agree on a culprit. We left the original half-hour time for the discussion, even if we feared a chatting platform would result slower than face-to-face conversation since typing and reading typically take more time than speaking. While chatting, the time still available is visible on top of the screen, aided by the by now usual red scrollbar.

On the right side of the screen, we placed the chat instructions and ensured that the user was clearly informed about which icon would represent them in the chat. The roles *A*, *B*, and *C* correspond to the three versions of the original study and are color-coded to enhance user clarity. To follow standard chat conventions, when a user types a message, it appears on the right, while messages from other participants appear on the left. To further distinguish the moderator, we highlighted their messages with a different background color and a black "M" icon.

This Page has two versions, depending on the condition it was run on. In the experimental condition, since the

moderator is present, some additional text is added to explain some of its features.

During development we noticed that some messages were not visible to the Azure API due to policy conflicts¹; as the central topic is a Murder Mystery messages would often regard topics such as weapons and bodies, meaning the GPT queries would not get a proper reply. To counteract this we had the moderator ignore some messages, we chose to alert the user by placing a red dot next to them (as visible in the Figure 6a). Naturally, both this feature and the related text warnings are not present in the condition without the moderator.

Next to the chat, there is a vote button. When pressed, a pop-up allows the user to choose between the three suspects. Once a suspect is selected, the user is directed to a screen where they wait for the other participants to cast their votes. At this point, the user can either close the pop-up to cancel their vote or wait for the other two participants to vote and see the results. A vote is considered successful if all three participants cast their votes and reach a consensus, in which case, participants are sent to the final Page. If a consensus is not reached, the participants are returned to the chat with a warning that the vote failed.

If the participants fail to reach a consensus before time runs out, all participants will receive a final vote pop-up. This pop-up will collect their votes and, regardless of whether a consensus is reached, will send them to the final Page.

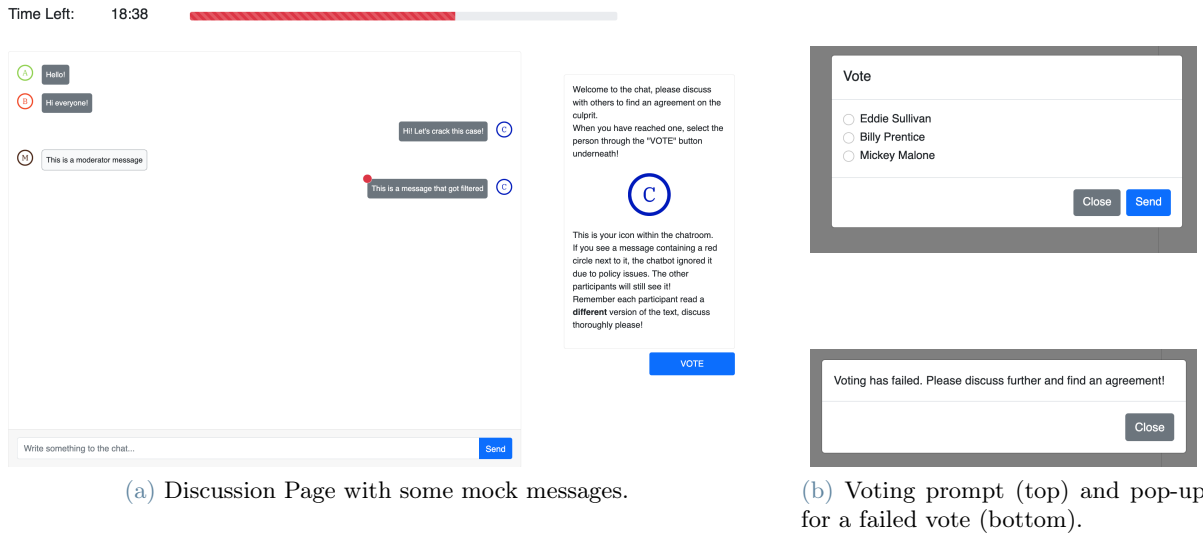


Figure 6: Snapshots from the *Discussion Page*, the study's seventh Page.

The final Page is the *Feedback Page*, which serves to gather the user's general opinions about the study for future improvements. This Page has a different purpose than its original counterpart, as we are not focused on whether participants felt the information they received was sufficient. Instead, our primary interest lies quality of the study: for example, we were interested in how fair the time allocation was for each task. We asked participants to rate, on a scale of 1 to 5, how they felt about the time provided for the *Reading*, *Questionnaire*, and *Discussion* Pages.

Additionally, we sought general feedback, asking two other questions about whether any technical issues arose and what general suggestions participants had for improving the study.

If the study was conducted with the experimental condition, two additional questions were included. The first asked participants about their experience with AI, while the second gathered their general thoughts on what worked well and what didn't regarding the moderator.

3.2.3 Data Collection and Analysis

In the original study, after all data was gathered, external coders were recruited to analyze the recordings of the discussions. This allowed them to accurately extract key values, such as the number of critical and non-critical clues. Each discussion recording was coded by three different individuals to account for the subjective nature of the coding process. Since the instructions provided to the coders were also available to us, we used the same coding criteria as in the original study. However, to save time, we opted for a single researcher to perform the coding task instead of having three outsourced coders.

¹see Microsoft Official Documentation (azure-content-filter)

If multiple coders listened to the recordings piece by piece and noted which information codes were mentioned, our researcher, in contrast, read all the messages and recorded if any information was present. The specific elements noted included critical clues, non-critical clues, inferences, and information requests. An inference is an assumption made by participants based on the available information, leading to a new understanding. Information requests are questions about the text.

3.2.4 Disconnection

Handling disconnections is a crucial difference from the original Strasser’s protocol and was a critical issue during our study. Given how Prolific operates, we could not guarantee that all participants would complete the study in its entirety. Participants were free to *return* the study at any time, which is an official Prolific practice indicating a user’s intention to stop participating. We also could not fully rely on participants to return the study properly; in fact, we observed instances where participants left during the reading stage without officially returning it. This could lead to scenarios where, for example, two out of three participants were left waiting indefinitely for a response from the third participant, who would never return.

To address this issue, we tracked participants’ active connections throughout the study by implementing a periodic heartbeat-like signal sent from the participant’s JavaScript to the server. However, during development, we observed issues where minimizing the study would cause the JavaScript to stop running. To mitigate this, we explicitly ask users to keep the study open at all times on the *Information Page*.

When a user fails to send heartbeats for 3 consecutive minutes, he is marked as "Disconnected". For the *Group Formation Page* and the ones previous to it, this is not an issue as no group is formed yet and there is no repercussion to a user leaving. On the other hand, this does not stand for further Pages as the 3-person group is already formed, and a disconnection results in 2 participants way ahead in the study. Instead of removing a disconnected user from the group and waiting for someone else to catch up with the remaining participants, we opted to keep the user marked as disconnected while ending the study for the other two participants. We chose not to replace the disconnected user with someone new because this could result in long wait times for the remaining participants, who would need to wait for the new participant to catch up. Such delays would likely lead to frustration and cause the remaining participants to leave as well, potentially resulting in a "live-locked" group where new participants join only to wait for previous participants to return.

Additionally, we would need to compensate the remaining participants for their waiting time, which could be costly. Therefore, the more cost-effective approach is often to minimize losses by paying participants for the fraction of the time they spent. For example, if a participant spends 20 minutes in the study and a disconnection occurs while they are reading the documents, they would be paid £3, which is one-third of the total £9 per hour rate.

4. Technology

4.1. Architecture

For the front-end development, we utilized JavaScript paired with Bootstrap. Due to time constraints, we opted for a minimalist design approach rather than investing in a more polished visual aesthetic, as we determined that focusing on functionality would be a more efficient use of resources.

To develop the back end of the study, we used Django, a Python-powered framework that allows for easy and robust web application development. The main driving factors for this decision were Django’s built-in functions, such as the CSRF tokens and the ORM for efficient database management. These tools made the development process significantly smoother for most of the study, Django’s view-url scheme greatly facilitated the organization and division of pages.

The most challenging aspect was integrating the LLM in the chat, as several adjustments were necessary to ensure compatibility with Django’s framework, which is optimized for fast views and responses. The primary issue was handling notoriously slow calls, such as querying an LLM, which can be time-consuming. In our case, this was further compounded by the fact that the LLM, GPT-3.5, is hosted on Azure, requiring network-based queries rather than local ones, adding additional latency to the process.

4.1.1 Group Formation

The *Group Formation Page* (Figure 4b) serves as a loading screen to ask the user to wait for a group to be formed. Within our design, we decided to dynamically assign incoming participants to groups made of three versions (A, B, and C) that make up the *Hidden Profile*. To explain how the matchmaking is operated within the API, let's define u as the calling user, g a group, G as the set of all possible groups, $g_u \in G$ a specific group where u belongs. We defined `num` as a variable sent to the user to display how many other participants are in his group.

Let's then define the functions $N(g)$ as a function that returns the number of users in a group g , $UpdateTime(u)$, a function that updates the time flag for user u , $RemoveInactive(g)$ a function that removes inactive users from a group g , and $LoadNextPage(u)$ a function that loads the following page for u . Lastly, we define $Add(g, u)$ a function that adds user u to a group g , and $create(u)$ a function to create a new group $y \in G$ and adds u to it.

Algorithm 1 Group Formation API algorithm

```
1: UpdateFlag( $u$ )
2: if  $\exists g_u \in G$  then
3:    $num := N(g_u)$ 
4: else
5:   if  $\exists g \in G$  such that  $N(g) < 3$  then
6:     RemoveInactive( $g$ )
7:     Add( $g, u$ )
8:      $num := N(g)$ 
9:   else
10:     $y := Create(u)$ 
11:     $num := N(y)$ 
12:   end if
13: end if
14: if  $num == 3$  then
15:   LoadNextPage( $u$ )
16: else
17:   return  $num$ 
18: end if
```

It must be noted that the frontend JavaScript schedules an API call every second, allowing *UpdateFlag()* to accurately track which users are currently on the page, down to the second. The function *RemoveInactive(g)* checks the flags of all participants in a group g only when a new participant is being added, this approach significantly reduces the load on the database, as it avoids performing the check at every API call.

This is important because the only reasons the JavaScript might fail to call the API correctly are if the user has closed the study or is experiencing severe connection issues, both of which should prevent the user from continuing.

4.1.2 Conversational Interface

When developing the chat, we needed to find a way to keep the front end constantly synchronized with the back end so that users could see new messages as they arrived. The software engineering's proper design pattern would have been to establish a socket connection. However, given that Prolific sometimes has unreliable participants [27], we were concerned that connection issues could cause the socket to close frequently, requiring us to recover and re-open it; a task that would be challenging and time-consuming. Therefore, we opted for a socket-less solution by implementing a JavaScript timer on the front end that polls the server every second to maintain synchronization. This approach is the same as the one used on the *Group Formation Page* and is commonly employed throughout the study whenever synchronization of the three users is required. In the case of the *Discussion Page*, the front end requests new messages from the server every second, which the server then provides. The server tracks which messages are new for each user using timestamps; each participant has a Timefield in the database called *last_message_request*, which is updated as queries proceed. When the front end receives the response to its previous call, it parses the received messages and appends them to the JavaScript DOM, making them visible in the chat.

Once a call is made, the server retrieves all messages that belong to the user's group and were sent between the time of the call and the user's *last_message_request* time. The server then returns these messages to the

user and updates the *last_message_request* field. This back-and-forth communication between the client and server is usually quite fast, but a slight delay between typing a message and seeing it appear in the chat was still noticeable. To counteract this and make the chat feel more responsive, we implemented a 'preview' feature. When the user types a message and presses send, before a POST is made to the server API that will save it, the JavaScript appends the message object with a 'preview' tag to the DOM. Once the user receives an update on all sent messages, the preview messages are removed from the DOM to make way for the actual messages, including the one the user sent. This also ensures that the chat always maintains the correct chronological order.

4.1.3 AI moderator

We will refer to our implementation of the LLM-powered moderator as the *AI Moderator*. The first and most straightforward solution we considered was to call an LLM for each message received by the server's API. While this approach appeared simple, it presented two significant flaws. Firstly, invoking the LLM within the same Django view responsible for saving a message would require waiting for the LLM call to complete, resulting in a slow API response—something Django is not optimized for. Secondly, and more critically, if the processing were to take place in the same API that receives the message POST the user sending the message would be stuck waiting for the LLM call, and thus the POST to be done. This would mean no further messages could be sent or received while the LLM call was ongoing, effectively making users stuck for some time after sending a message.

During initial testing, we experimented with a design that triggered a call for every received message, but we encountered cases where the LLM duplicated the same message. For instance, given a critical message (M1) requiring intervention and a less important message (M2) expressing agreement, if M1 and M2 were sent in quick succession, the moderator would respond to M1 twice, as it could not account for its previous intervention.

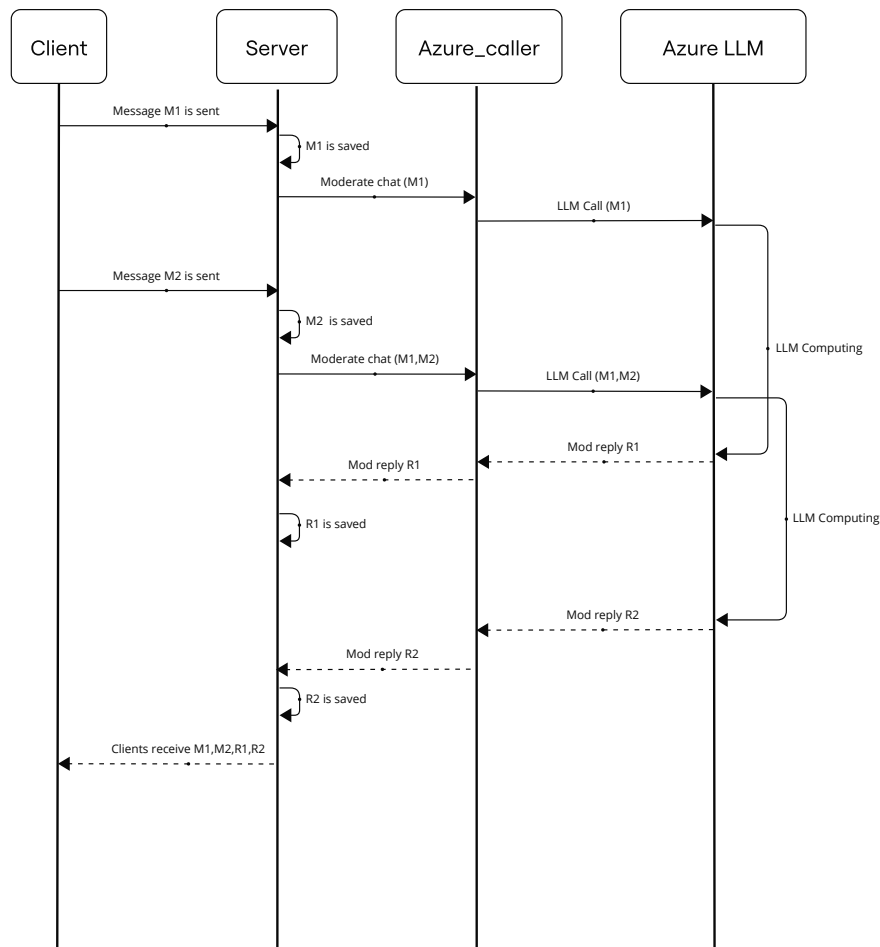


Figure 7: Sequence diagram portraying the first solution, and the double texting problem (R1,R2).

To resolve these issues, we detached the LLM call from the message-receiving process by creating asynchronous tasks using Asyncio, a Python library that allows concurrent code execution via the `async/await` syntax [29]. We developed a Django command called *Azure_collector*, specifically tasked with invoking the LLM when necessary. We also defined three states in which a group could exist:

- *Idle* (0): The default state of the group, indicating that no action is required.
- *Request* (1): Indicates that a new message requiring moderation has been received, necessitating an LLM call by the *Azure_collector*.
- *Handling* (2): Indicates that the group is currently undergoing an LLM call made by the *Azure_collector*.

The first group state, *Idle*, is the default state for all groups. When a group receives a message through its API, if it is in the experimental condition and currently in the *Idle* state, its state changes to *Request*. The *Azure_collector* command, which runs as a service, monitors for groups in the *Request* state. When it detects one, it changes the group's state to *Handling*, and invokes the LLM. After the call is completed, the group's state reverts to *Idle* until a new message arrives, and the cycle begins anew.

Notably, if a message arrives while the group is in the *Handling* or *Request* state, it will not trigger a new LLM call. This design decision ensures that only one LLM call can be active at a time for each group, effectively addressing the double texting issue reported with the earlier design (see Figure 7). To visualize our design see Figure 8.

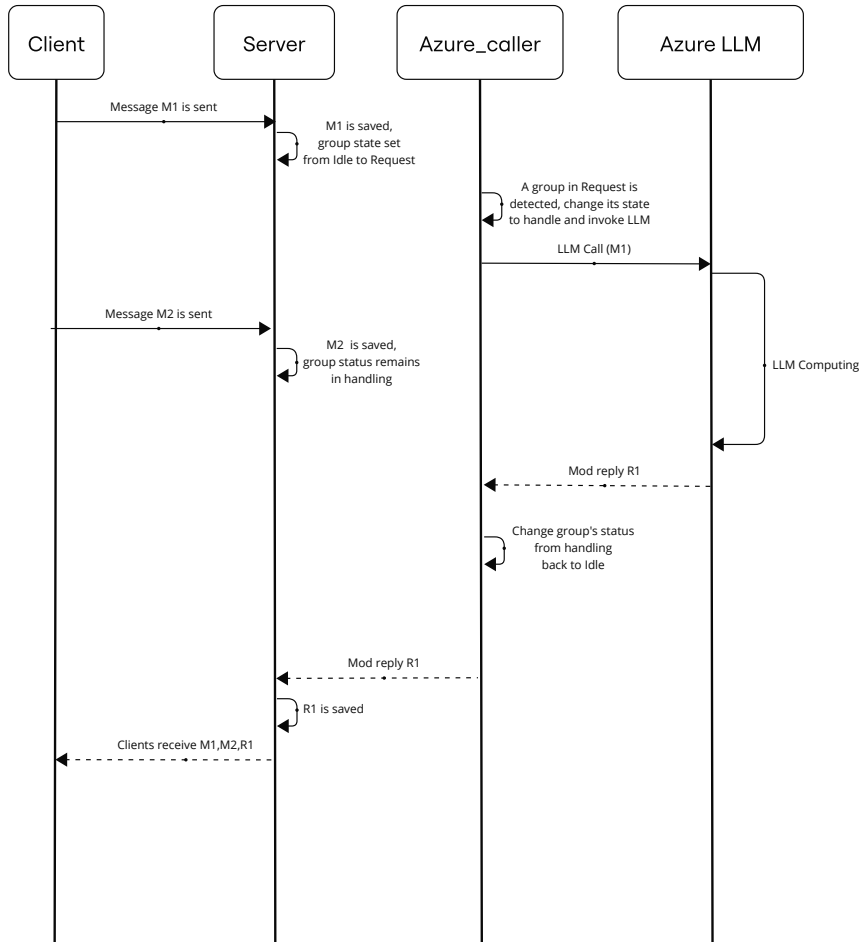


Figure 8: Sequence diagram portraying how the implemented solution works.

Another design choice we considered was canceling an ongoing LLM call whenever a new message was received. This approach would allow the LLM to stay more in sync with the conversation when responding. For example, if a message M1 triggers an LLM call and, before the call is completed, a second message M2 arrives, the first

call would be canceled and a new call would be made to include both M1 and M2. In theory, this method should be more accurate than our current approach, as the LLM's response would account for both M1 and M2, rather than being based solely on M1.

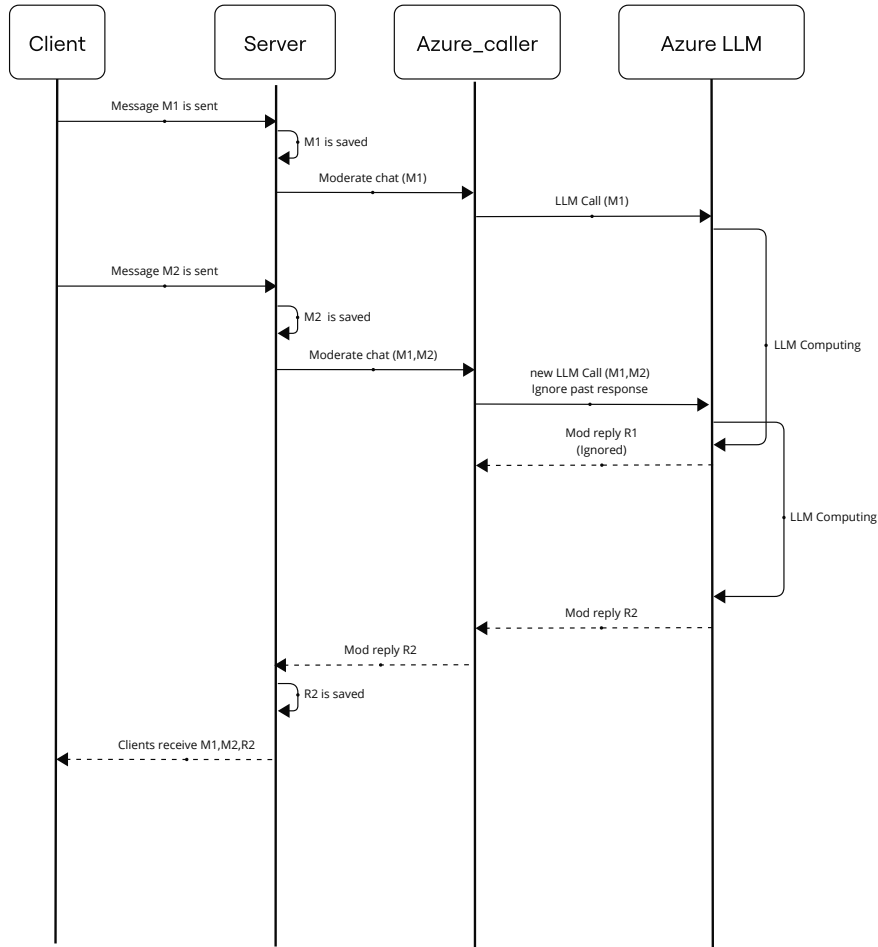


Figure 9: Sequence diagram representing the "delayed response" problem.

The core flaw of this approach becomes apparent in a lively chat: if multiple successive messages arrive, the moderator will continually delay its response until the barrage of messages ceases. This could result in missed key intervention points and a significant delay in the moderator's response. We further argue that our solution better imitates a real-life participant who starts typing upon receiving the first message and may not have the chance to read M2 in time.

Thanks to prompting, once the LLM call is done it should return a JSON object, which is then parsed for its content. If the LLM hallucinates and does not return a valid JSON, the reply is ignored, and the group state is reset to *Idle*. We decided against repeating the query in hopes of receiving a correctly formatted JSON response, as by the time the call was completed, additional messages might have arrived, further complicating the design of the solution. Moreover, there was no guarantee that the hallucination wouldn't persist indefinitely. We were also aware of an OpenAI function that automatically returns the LLM's answer in JSON format, but we deliberately chose not to use it. Utilizing this feature would limit us to OpenAI's models and preclude the option of using other LLMs in the future. Therefore, we intentionally left the JSON formatting to the LLM itself.

If the JSON is correctly parsed, its content is saved as a message the moderator sends. Users will then receive this message along with other messages as explained earlier in Section 4.1.2. It's important to note that a new message from the moderator does not trigger another call from the *Azure_collector*, as only messages received through from users can change a group's state. We intentionally designed it this way to prevent moderator messages from causing a loop where, in the event of hallucinations, the LLM might start conversing with

itself and take over the chat.

The LLM we used was GPT-3.5, hosted on Azure cloud services. We specifically chose Azure because it allows for a controlled model that does not train on your data. Before the testing began, we queried the LLM regarding the murder mystery transcripts and characters, and nothing relevant surfaced. This confirmed that the version of ChatGPT used was not aware of the culprit’s identity and, therefore, could not introduce bias during the discussion. Had we used public versions of GPT through the API, we could not guarantee that it would not train on previous conversations. Additionally, relaying participants’ information to a third party, such as OpenAI, would violate GDPR (EU) and UK data protection policy regulations. The University Ethics Commission formally approved the research (UCLIC_2022_004_costanza).

During the study, we discovered that Azure has a content policy filter that cannot be disabled. This filter classifies all incoming LLM calls and detects instances of violence, racism, or other unacceptable behavior. Since the discussion topic was a murder mystery, messages frequently contained elements of violence, leading the system to flag them and throw exceptions. Unfortunately, we found no effective workaround for the filters, as they were integrated into Azure rather than the LLM itself. This meant that prompt engineering techniques, such as gamification or jailbreaking [8], were ineffective. Our solution was to simply ignore messages that the system flagged as inappropriate, alerting the user with a red dot for clarity (see Figure 6a). Additionally, during testing, we did not identify any specific words that consistently triggered the filter, so we were unable to "catch" and replace troublesome words before invoking the LLM.

4.2. Prompt Engineering

Instead of using techniques like Retrieval-Augmented Generation (RAG) or fine-tuning, we opted for prompt engineering. We ruled out fine-tuning due to the lack of an available training dataset, and RAG was unsuitable because it significantly increased latency, negatively impacting the user experience. Thus, we focused on modifying the LLM’s behavior through iterative prompt design. We present these iterations and the rationale behind them in three distinct phases, with a pilot study conducted at the end of each phase. Findings from each pilot were then incorporated into the subsequent phase.

4.2.1 Phase 1

Our goal was to test whether a very simple LLM moderator could induce changes in a multi-user chat. We were not looking for a specific solution to the task but rather a generalized version of a baseline moderator. Additionally, we aimed to explore the LLM’s turn-taking abilities, as, to our knowledge, no research has featured a completely independent LLM in this context. Therefore, we decided to let the LLM control both *what* to say and *when* to say it. To achieve this, we designed our prompt to ensure that the output was always a JSON object containing two fields: *INTERVENE* and *TEXT*. The *INTERVENE* field was a boolean value indicating whether the LLM decided to intervene. This determined the *when*, as we would only register a moderator message in the chat if this field was set to true. The *TEXT* field addressed the *what*, containing the text the LLM would use to intervene. If the *INTERVENE* field was false, meaning the LLM chose not to intervene, we instructed the LLM to include a reason in the *TEXT* field for not intervening, although this information was not used in the actual study. We hoped this approach would provide some level of explainability for the LLM’s behavior.

Our initial idea for determining the optimal *when* to intervene was to consult past literature for guidelines that could define the role of a moderator in a similar context. Unfortunately, most of the moderator or facilitator roles described in the literature were not suited to our task, as they originated from very different settings, as previously discussed in Section 2. Among these, the most promising source was Kruger’s work on focus groups [23], which, among various approaches, clearly defined the *challenger* role, identifying the more general qualities we sought in a moderator. The challenger role was described as:

This moderator challenges participants to explain, amplify, and justify their ideas and actions. Occasionally, this moderator will even position one participant against another participant who seems to have an opposite view. Challengers need good timing and effective group skills to avoid alienating the participants. Successful challengers can sometimes bring to the surface opposing points of view that may remain undisclosed with other strategies.

This approach did not yield satisfactory results during initial testing, so we decided to create our own guidelines by consulting the LLM directly. We prompted GPT-3.5, GPT-4, and Bing AI with the following question:

As a highly intelligent AI system, you are moderator (M) in a conversation between three participants (A, B and C) who need to agree on a decision. Your job is to guide the conversation, without being too intrusive. A good moderator intervenes only when the timing is right. Please list all the situations in which you would intervene.

The phrase, "*As an intelligent AI system*", is adopted from literature on prompt engineering to improve overall performance [15]. Note how, within the prompt, we never notice how the task involves a murder mystery, we define it as a general decision, this is because we want to look for a general moderator that does not have particular rules for our task. What resulted, out of the calls of the respective LLMs, were some general rules that the LLM suggested as course of action. We took all of the rules, that happened to be quite similar from each other, and selected only the common ones among the calls. Here were the rules we ended up using for our prompt:

1. If one or more participants are dominating the conversation and not allowing others to speak.
2. If a participant is being disrespectful or using inappropriate language.
3. If there is a misunderstanding or miscommunication between participants that is preventing them from reaching an agreement.
4. If the discussion is going in circles and no progress is being made.

With these guidelines in mind as baselines, we needed to develop a way to test and validate the potential prompt. For this initial phase of testing, we selected conversations from the no-moderation condition that had already taken place. Each of these chats contained n messages, where each message is denoted by m_i , with i ranging from 1 to n , the total number of messages in the chat. For each message m_i , we queried the LLM with that message and its precedent messages: $m_{i-1}, m_{i-2}...m_1$. At each iteration, we saved the LLM reply. The results were n different chat snippets, each with its own LLM response. By varying the prompt used, we could test how a first reply to each chat would look. At this stage, we chose not to include the moderator's message in the previous iterations, as doing so would inevitably alter the conversation's trajectory, making any non-live test unreliable.

Testing unreliability proved to be one of the two biggest challenges during development. The only reliable way to test a prompt was to run the study with real participants, which would have meant paying all of them, incurring a cost of £27 per tested prompt. For both time and economic reasons, iteratively improving the prompt through live testing was not feasible. The second major challenge, closely related to this, was the validation of the prompt. Unfortunately, despite our literature review, we found no established metric to evaluate the moderator's performance in a test. While such metrics are readily available when a group completes the entire study—where clues, results, and statistical analysis can usually indicate whether the moderator had a positive effect—this is not the case in testing. In testing, the chat and its clues are predetermined, so changes to the prompt must be based largely on intuition. Since our goal was to create a generic implementation, we didn't aim for a particularly refined prompt anyway, but the few adjustments we made were solely based on intuition given the simulations.

After some testing with the method explained earlier, we found that the guidelines we extracted from the LLMs performed reasonably well in terms of the content of the interventions. However, the *when* factor was difficult to evaluate, as the moderator tended to always intervene as the chat progressed. We believe this occurred because the LLM would see a large number of messages, and among them, some would inevitably require intervention. We considered cutting the chats into smaller snippets to test the *when* factor more effectively starting from, for example, $i=9$ instead of 1, and observing messages like $m_9, m_{10}, m_{11}...$ and so on. However, we ultimately decided that trying the prompt out in real-time would be the best decision.

We thus proceeded to run our first pilot with the moderator, through some personal acquaintances. The full prompt we tested was:

As a highly intelligent AI system, you are the moderator (M) in a conversation between three participants (A, B and C), about a murder mystery. Your job is to guide the conversation, without being too intrusive. A good moderator is one that does intervene, but only at the right time and not too often. Some guidelines for intervention are:

1. *If one or more participants are dominating the conversation and not allowing others to speak.*
2. *If a participant is being disrespectful or using inappropriate language.*
3. *If there is a misunderstanding or miscommunication between participants that is preventing them from reaching an agreement.*
4. *If the discussion is going in circles and no progress is being made.*

Please always respond via a JSON file that contains a flag INTERVENE and a TEXT field. In case you, as the moderator, have to intervene within the chat conversation, set the INTERVENE flag to true and add your answer in the TEXT field. Make sure both fields are always distinct and INTERVENE is only true or false. If as a moderator you don't intervene, set INTERVENE to false and place in TEXT your reasoning.

This prompt was run with the first solution, displayed in Figure 7. During our first trial, we noticed the *AI Moderator* tended to repeat itself after some time in the chat had elapsed and happened to double text some times. As expressed earlier, we attributed the double texting to a quick succession of two messages M1 and M2, with two following replies to M1. We also noticed that, as the chat went on, the moderator started intervening more often, to the point that by the end of the chat, the bot was replying basically every message.

4.2.2 Phase 2

With so much to tackle on the table, we decided to implement a new way of testing, trying to only fix systematic problems instead of particular ones. Again, as we wanted to test a moderator for a generic task, we were careful not to fit the moderator around to the murder mystery. For example, we noticed that during the first pilot, the LLM would often hallucinate a non-JSON response, resulting in no message for that interaction. This kind of issue was generic to the moderator function, and not related to the particular task at hand. To test these issues we implemented a new algorithm, that given the test chat would allow us to tweak the prompt and see how it would impact a potential chat.

Let's define the empty chats S as the simulation chat and C_s as the stripped original chat. The original moderated chat is C , containing n messages, where each message is denoted by m_i with i ranging from 1 to n . We then define some functions. Firstly, $IsModerator(m)$ is a Boolean function that returns true if the message m was written by a moderator. $AddMessage(C, m)$ appends the message m to the end of chat C . Lastly, let's define $QueryLLM(m_j)$, a function that returns the LLM's response r to the chat composed of the previous chat history $m_j, m_{j-1}, m_{j-2} \dots m_1$. Finally, let's define $Parse(r)$, which takes an LLM response r and parses it from JSON into a message m . If the JSON is not parsable, it returns -1.

Algorithm 2 Algorithm for simulations

```

1: for each message  $m_i \in C$  do
2:   if  $IsModerator(m_i) == \text{false}$  then
3:      $AddMessage(C_s, m_i)$ 
4:   end if
5: end for
6: for each message  $m_j \in C_s$  do
7:    $AddMessage(S, m_j)$ 
8:    $r = QueryLLM(m_j)$ 
9:   if  $Parse(r) == -1$  then
10:    Print("Error: JSON not parsable")
11:   else
12:      $AddMessage(S, Parse(r))$ 
13:   end if
14: end for

```

This new algorithm creates a simulation chat S by stripping all moderator messages from the original chat C and replacing them with new ones generated from a different prompt. It is important to emphasize that this is only a simulation and cannot be a reliable indicator of how the chat will behave, as the moderator cannot influence and steer the conversation that has already occurred. We argue that since the moderator was present during the original chat, a new moderator would make similar decisions. Therefore, this simulation can be useful in highlighting the similarities that emerge with small changes in the prompt.

For example, through these simulations, we tested ways to improve resistance to JSON hallucinations. During the initial run, the model frequently outputted invalid JSON. To try and solve the issue, we reinforced the prompt by including two examples of correctly formatted JSONs from previous responses. Additionally, we noticed that the prompt still referenced the task by stating "*about a murder mystery*". We were concerned that this might fine-tune the model towards solving such problems, thereby undermining the generalizability of the solution.

In subsequent simulations, we removed this reference to the task and replaced it with *"who need to agree on a shared decision"*. As the model's responses did not significantly change with this alteration, and the example JSONs appeared to reduce hallucinations, we decided to run another test pilot. In this pilot, we changed the architecture from the naive one (Figure 7) to a state-based one (Figure 8), while keeping the previous prompt with these modifications.

As a highly intelligent AI system, you are the moderator (M) in a conversation between three participants (A, B and C), who need to agree on a decision. Your job is [... - see above prompt]

Make sure both fields are always distinct and INTERVENE is only true or false. If as a moderator you don't intervene, set INTERVENE to false and place in TEXT your reasoning. Here are some examples of JSON files: {"INTERVENE": false, "TEXT": "The conversation is developing organically, I will not intervene."}, {"INTERVENE": true, "TEXT": "It appears both B and A have expressed suspicions about the same suspect. To keep the conversation balanced, I recommend inviting C to share their thoughts as well."}

4.2.3 Phase 3

In terms of the tackled problems, the pilot was largely successful: the occurrence of JSON hallucinations decreased significantly, and the new architecture effectively eliminated the issue of double successive moderator messages. Furthermore, the absence of the *"murder mystery"* part of the prompt did not seem to throw the bot off-guard, it quickly caught on that the users were leading the investigation.

However, some problems remained, notably the frequency and quality of interventions, as the *AI Moderator*, after some messages, started outputting the same summary of the chat over and over again. We found out this was caused by the way we were inputting the OpenAI API, we were inserting all of the chat within the context, without distinguishing between agent and user messages. We thus changed the way we used the API, as detailed in Figure 10.

```
Old Api Call Python

input = ""

messages = await self.get_messages(group)

for message in messages:
    role, timestamp, text = await self.mex_unpack(message, group)
    input += f"{timestamp} - {role}: {text}\n"

response = await client.chat.completions.create(
    messages=[
        {"role": "system",
         "content": sys_prompt},
        {"role": "user", "content": input}],
    model=deployment_name,
    max_tokens=120
)
```

(a) Old API

```
New Api Call Python

inputList = []

inputList.append({"role": "system", "content": sys_prompt})

for message in messages:
    role, timestamp, text = await self.mex_unpack(message, group)
    if role == 'M':
        interveneJson = """{"INTERVENE": true, "TEXT":\n"" + \
            str(text)+"\n"}"""
        inputList.append(
            {"role": "assistant", "content": interveneJson})
    else:
        inputText = f"{role}: {text}"
        inputList.append({"role": "user", "content": inputText})

response = await client.chat.completions.create(
    messages=inputList,
    model=deployment_name,
    max_tokens=120
)
```

(b) New API

Figure 10: Old usage of the API (left) Vs. New one (right).

Before the change, the variable *input* contained all of the previous messages as a single string. After the change, the API was provided with the input in the form of a list. The variable *inputlist* is a list of JSONs differentiating messages from users and previous messages from the moderator, through the *"role"* field.

Through simulations using the newly obtained chat data and this new method of utilizing the API, we observed that the *AI Moderator* stopped repeating its messages after some time. We hypothesized that the improved API call contributed to this change, as the content of the replies shifted from repeating the same summary to more accurately reflecting the chat's content. We speculated that feeding the entire chat as a single message might have caused the LLM to 'lose' the meaning of the text due to the excessively long context.

During testing, we began to notice a new tendency in the *AI Moderator*'s replies following the introduction of the new prompt. Specifically, it started responding in the third person, with many replies commencing with

phrases like 'it seems.' Additionally, instead of addressing participants directly, the bot often offered suggestions using statements such as 'I suggest inviting B to...'.

We quickly realized that the cause of this behavior was the example JSON we provided, particularly the example message within it. The message was randomly selected from a test, with the sole intention of demonstrating the correct JSON formatting. However, it became apparent that the LLM interpreted the example not only as a formatting guide but also as a template for content, which influenced its style and replies. Since we sought a general solution, we proceeded to delete the content of the example messages, both true and false, to avoid inadvertently prompting the LLM. For future studies interested in developing a tuned moderator, few-shot learning via this approach appears to be a promising option. But for our study, we ran more tests changing the prompt as such:

*...Here are some examples of JSON files: {"INTERVENE": false, "TEXT": "*message to send*"}, {"INTERVENE": true, "TEXT": "*message to send*"}*

We ran some tests with this change, and it seemed as if the bot started addressing people directly again, for example with "C, could you elaborate on why you suspect Billy Prentice specifically?". We decided to run a final pilot, utilizing the the following prompt:

As a highly intelligent AI system, you are the moderator (M) in a conversation between three participants (A, B and C), who need to agree on a decision. Your job is to guide the conversation, without being too intrusive. A good moderator is one that does intervene, but only at the right time and not too often. Some guidelines for intervention are:

- 1. If one or more participants are dominating the conversation and not allowing others to speak.*
- 2. If a participant is being disrespectful or using inappropriate language.*
- 3. If there is a misunderstanding or miscommunication between participants that is preventing them from reaching an agreement.*
- 4. If the discussion is going in circles and no progress is being made.*

*Please always respond via a JSON file that contains a flag INTERVENE and a TEXT field. In case you, as the moderator, have to intervene within the chat conversation, set the INTERVENE flag to true and add your answer in the TEXT field. Make sure both fields are always distinct and INTERVENE is only true or false. If as a moderator you don't intervene, set INTERVENE to false and place in TEXT your reasoning. Here are some examples of JSON files: {"INTERVENE": false, "TEXT": "*message to send*"}, {"INTERVENE": true, "TEXT": "*message to send*"}*

Since the pilot yielded results that we deemed satisfactory, we utilized this prompt for the actual study. We considered the final prompt as general enough to represent a moderator in a decision-making task, as it contains no references to the mystery.

4.2.4 Summaries

During the development of the prompt, we were aware of the finite nature of the LLM's context. As we examined the shortcomings of the second tested prompt, we considered ways to shorten the context that the LLM received. We conceived of using summaries as a method to reduce the LLM's context while allowing chats to extend beyond the base GPT context length. Consequently, we developed another algorithm for testing, which we believe could be beneficial for future studies. This algorithm is a modification of the algorithm described before (i.e., Algorithm 2), so we will retain the same base definitions.

We introduce a new variable, *interval*, defined as an integer representing the number of messages between each summary. After every *interval* messages, a summary *summ* of the latest messages is generated, through the *Summarize*(m_j) command. This summary is then appended to a string containing all previous summaries, referred to as *SSummaries*. The *SSummaries* string starts empty and gradually accumulates content through the function *AppendSummary*(*summ*).

We also have to redefine the *QueryLLM*($m_j, interval, SSummaries$), function to account for the new variables. This function provides the LLM with a context composed of the previous summaries *SSummaries* along with the latest *interval* messages, specifically the messages $m_j, m_{j-1} \dots m_{j-index}$. The function still returns a response *r*.

Algorithm 3 Algorithm for simulations using summaries

```
1: for each message  $m_i \in C$  do
2:   if  $IsModerator(m_i) == \text{false}$  then
3:      $AddMessage(C_s, m_i)$ 
4:   end if
5: end for
6: for each message  $m_j \in C_s$  do
7:    $AddMessage(S, m_j)$ 
8:   if  $(j \% index) == 0$  then
9:      $summ = Summarize(m_j)$ 
10:     $AppendSummary(summ)$ 
11:  end if
12:   $r = QueryLLM(m_j, index, SSummaries)$ 
13:  if  $Parse(r) == -1$  then
14:     $Print("Error: JSON not parsable")$ 
15:  else
16:     $AddMessage(S, Parse(r))$ 
17:  end if
18: end for
```

To perform summaries, we used an additional ChatGPT call; however, other LLMs or tools (such as a fine-tuned BERT) are also available. We conducted tests with this algorithm using $index=10$, given that the test chats were approximately 80 messages long. During testing, we observed that the use of summaries did not provide a significant benefit. Furthermore, upon reviewing the summaries generated by GPT, it appeared that some information was occasionally omitted.

Additionally, defining and tuning an appropriate prompt for these summaries introduced complexity, which was counterproductive to our goal of utilizing a simple, off-the-shelf LLM moderator. Despite these issues, we report this method as it could still be valuable for future testing with live participants and might be improved beyond our initial implementation. Moreover, approaches like this are necessary for managing longer chat lengths, although this was not an issue in our case, as all chats (including moderator messages) could fit within the LLM’s context window.

5. Results and Discussion

5.1. Participants

We ran the study on Prolific a total of 20 times, 10 times per condition (control and experimental conditions). Each participant was awarded £9 for 1 hour of their time. The data for chats without a moderator are reported in Table 2, while the data for chats with a moderator are presented in Table 5. The 60 participants were recruited from Prolific, and we filtered for users with good English proficiency and no lexical impairments, as a quick and thorough understanding of the task was essential for the study’s success. The participant’s Age and Sex are reported in the Table 1 below.

Condition	Age Mean	Age Std	Male	Female	N.A.
Moderator	36.36	10.88	14	15	1
No Moderator	29.08	8.31	17	12	1

Table 1: Table reporting distribution of users through conditions.

5.2. Variables

The first and most important metric was the correct solving of the murder mystery, we analyzed who each group appointed as the culprit, and reported who their agreed choice was. The options were Eddie, Billy, and Mickey, with Eddie being the culprit and thus the right answer. We still report the results of the votes if when a consensus wasn't reached.

As previously highlighted in Section 3.1, the study distinguishes between **critical** and **non-critical** clues. To leverage them, we coded all of the 20 discussions using the original coding material used by Stasser. We analyzed the sheer number of non-critical and critical clues within each chat. In Tables 2 and 5, we also report the **Total info** as the sum of the number of critical and non-critical clues. As in the original study, we analyze the **discussion focus** defined as:

$$Discussionfocus = \frac{Critical}{Totalinfo}, \quad (1)$$

The discussion focus is a percentage representing the amount of discussion spent on focal clues.

During the coding process, we counted the number of explicit requests for information, denoted as **Info request**. We also counted the number of instances in which users made inferences, reporting this under the row labeled **Inference** in Tables 2 and 5. Both of these values were weighted against the total number of messages sent by users in the chat, to account for the high variance in chat length. These are displayed as percentages under **Info request/Total** and **Inference/Total**. We also report the **Discussion Time**, defined as the number of minutes the three users required to reach a consensus. In cases where the discussion time exceeded 30 minutes, users voted regardless of reaching a consensus, as explained in Section 3.

We also analyzed the number of messages the moderator and users sent, and called them respectively **Mod Messages** and **User Messages**. A sum of them is available as **Total Messages**, which is equal to the number of user messages in the control condition.

We also introduced a new metric called *intervention rate*, to properly quantify how often the *AI moderator* intervenes from chat to chat. In our framework, a moderator response is triggered only in response to a user message. Therefore, if a moderator intervenes at every possible opportunity, their number of messages would be equal to the number of messages sent by users, constituting half of the total messages. Conversely, if the moderator never intervenes, the total number of messages would correspond solely to the number of messages sent by users. We define the *intervention rate*, denoted by I_r , as the ratio of moderator messages to user messages:

$$I_r = \frac{Mod\ Messages}{User\ Messages} \quad (2)$$

A moderator with $I_r=1$ has a 100% intervention rate, meaning they intervene after every user message, while a moderator with $I_r=0$ never intervenes. It is worth noting that due to the structure of our application, as explained in Section 4.1.3, very short or rapid messages may not trigger a response. As a result, the recorded I_r values may be slightly lower than the actual intervention capability of the *AI Moderator* in a different framework.

Lastly, in an effort to explore discussion balance, we employed the *Gini Index*, a measure of statistical dispersion often used in economics to assess inequality in resource distribution. The index ranges from 0 to 1, where 0 indicates perfect equality (i.e., all actors have the same amount of the resource), and 1 indicates complete inequality (i.e., one actor possesses all the resources). The *Gini Index* formula is:

$$G_n = \frac{\sum_{i=1}^n \sum_{j=1}^n |x_i - x_j|}{2n^2 \bar{x}}$$

where x_i represents the resource amount for actor i out of a total of n actors, and $\bar{x}$ is the mean of the resource distribution. Our goal is to calculate the discussion balance in every group, we will utilize the number of messages sent by each user as a measure of contribution to the conversation. Thus, chats where a single user speaks significantly more than others should result in a G_n of 1 indicating a high imbalance. However, reviewing the literature, we know that the *Gini Index* should be adjusted for biases created by small sample sizes, according to the Delta's adjusting factor [13]:

$$G_n^{Adj} = \frac{n}{n-1} G_n$$

Thus, our final index, which accounts for the small sample size $n = 3$ is:

$$G_n^{Adj} = \frac{3}{2} \cdot \frac{\sum_{i=1}^3 \sum_{j=1}^3 |x_i - x_j|}{2 \cdot 3^2 \bar{x}} = \frac{1}{12} \cdot \frac{\sum_{i=1}^3 \sum_{j=1}^3 |x_i - x_j|}{\bar{x}}$$

We report the measures of the adjusted metric in Tables 2 and 5 under **Gini Adjusted**.

5.3. Control Condition (Without the moderator)

Group ID	1	2	3	4	5	6	7	8	9	10	Average
Voted	Billy	Billy	Billy	Eddie	Eddie	Billy	Billy	Eddie	Billy	Billy	Billy
Non-critical	6	5	23	26	9	7	3	13	1	7	10
Critical	0	0	2	3	1	0	0	3	0	2	1.1
Total info	6	5	25	29	10	7	3	16	1	9	11.1
Discussion Focus	0%	0%	8%	10%	10%	0%	0%	18.75%	0%	22.22%	6.93%
Info request	5	5	11	14	10	4	5	6	4	4	6.8
Info request/Total	12.50%	14.28%	10.47%	19.44%	20.80%	12.12%	17.24%	8.95%	26.66%	14.81%	15.73%
Inference	9	10	33	12	9	1	2	10	1	2	8.9
Inference/Total	22.50%	28.57%	31.42%	16.66%	18.75%	3.03%	6.89%	14.92%	6.66%	7.40%	15.68%
Mod Messages	0	0	0	0	0	0	0	0	0	0	0
User Messages	40	35	105	72	48	33	29	67	15	27	47.1
Total Messages	40	35	105	72	48	33	29	67	15	27	47.1
Discussion Time (mins)	9.1	7.31	27.85	26.23	9.77	3.57	4.55	15.69	14.45	9.94	12.84
Gini Adjusted	0.10	0.22	0.18	0.46	0.12	0.27	0.20	0.16	0.33	0.07	0.21

Table 2: Table with the results obtained from groups without the moderator.

The first thing to notice is the final votes, within the control condition only 30% of groups correctly guessed the culprit (Eddie). This percentage is far from the original study, which in the corresponding conditions correctly guessed the culprit with a percentage of around 65%. This may lead to the belief that there is a problem with the digitalization of the study, which does not translate the physical elements into the online version. But, by reading the transcripts and reviewing additional data, we argue that some fault may be attributed to how Prolific works. Its users are hasty, as they look forward to the compensation and sometimes disregard the success of the study, in exchange for a quick reward [27]. To back our theory up, we want to highlight some data collected through the study, that better explains the reason for our suspicions.

Since the reading time is shared between conditions, we considered data from both. Users had an average reading time of 24 out of the 30 available minutes. This suggests that, on average, users spent sufficient time reading and understanding the contents of the study. The results of the questionnaire page (see Figure 5b) further prove this hypothesis, by analyzing them we found that, out of the 60 users that partook in the study, 43 (71,67%) of them responded perfectly to all the questions related to the reading, meaning most of the users did pay enough attention to the transcript they read. However, due to the *Hidden Profile* condition, full attention from *all* participants in a group was crucial for ensuring they had all the necessary information for the task. Even one of the three participants missing their share of the information could negatively impact the study’s outcome. Unfortunately, 11 out of the 20 groups in the study had at least one participant who answered the questionnaire incorrectly.

We analyzed whether a group’s good score in this questionnaire correlated to a correct guess of the culprit. Performing χ^2 we found no statistical significance between the two ($p=0.3527$), further proving that attention to the reading alone was not enough to complete the task successfully, a productive chat was needed for a successful result.

We also wanted to test whether there was an issue with overall motivation, by running a biserial correlation between the users with perfect questionnaires and the number of messages they sent, we theorized an unmotivated user would not pay attention and would hastily reply to the chat to end the study as quickly as possible. To remove the effects of the moderator, we ran the biserial only in the no-moderator condition and found no statistical significance between the two ($\rho_b = 0.2433$, $p=0.1950$).

The following test we did was to correlate the number of messages sent by users in the group to the correct decision of the suspect. We started by running the biserial only in the non-moderator condition but found no statistical evidence ($\rho_b=0.3903$, $p=0.2647$). We then extended this to all groups, including the participants from the moderator condition, by excluding moderator messages. Within the full scope, we still didn’t find significance within the data ($\rho_b = 0.3973$, $p=0.0827$), this could still be in line with the original study, which affirmed that a discussion needed to be focused on the relevant topics to be successful. To measure the focus, we first considered the sheer amount of information shared, and in neither condition significant evidence was

found ($\rho_b = 0.2686$, $p=0.4528$ with the moderator; $\rho_b = 0.4972$, $p=0.1436$ without it).

In the original study, it was further argued that the success rate was higher if groups focused on critical clues, where the focus was defined as the number of times a piece of critical information is mentioned divided by the total amount of information. This translates in our case, when considering all groups their success rates are related to its *Discussion focus* ($\rho_b = 0.4507$, $p=0.0460$). It is thus confirmed that a focused conversation correctly leads to the solution. Still, in the moderator condition, when examining time in relation to the focus we notice no correlation ($\rho_b = 0.3352$, $p=0.1484$), meaning a longer chat may not necessarily mean more focus.

Group ID	Pilot1	Pilot2	Pilot3	Average_no_mod	Pilot_mod	Average_mod
Voted	Eddie	Eddie	Mickey	Billy	Eddie	Billy
Non-critical	17	22	23	10	21	12.7
Critical	7	6	2	1.1	4	1.6
Total info	24	28	25	11.1	25	14.3
Discussion Focus	29.16%	21.42%	8%	6.93%	16%	9.60%
Info request	18	16	17	6.8	22	10
Info request/Total	19%	12%	13%	15.73%	21%	16.70%
Inference	17	20	21	8.9	22	6.8
Inference/Total	18%	14%	17%	15.68%	21%	10.25%
Mod Messages	0	0	0	0	63.00	22
User Messages	97	135	126	47.1	104	57.90
Total Messages	97	135	126	47.1	167.00	79.9
Discussion Time (mins)	29.26	29.55	29.4	12.84	29.41	18.65

Table 3: Table comparing the internal pilots with the average online participants

With these statistics in hand, it becomes evident that the data from the original study did not fully translate to its online counterpart, as indicated by the low success rate. However, when we analyze the internal pilot tests conducted to evaluate the platform (see Table 3), we observe a completely different scenario. In the control condition, both groups guessed correctly, with much longer average discussion times and a significantly higher focus on the discussion. Additionally, all internal groups utilized the entire available discussion time, which allowed them to explore the case in-depth, scoring more than double the total information in both conditions. This highlights the major issue faced by the online participants. By reviewing the transcripts, we noticed that they tended to settle on the first available answer rather than thoroughly exploring other options. This tendency often led them to vote for Billy, the character framed by the story as the "obvious" culprit. If the online groups had engaged in more thorough discussions, they likely would have shared more information. This is further reinforced by the original study, which states that, on average, 26 pieces of information were shared. This data is much closer to the thoroughly discussed pilot studies than to the shorter online groups.

We analyzed the results of the *Questionnaire Page*, more particularly about who the users considered the culprit given the transcripts. We report the results in Table 4, in the *Questionnaire* row. Given the original paper's explanations, we were expecting the values to be evenly distributed, thus having around 20 votes for each suspect. By running a χ^2 test, we confirmed an uneven distribution among votes, skewed towards Billy ($p=0.0363$). This also translates to the final votes, which end up being largely towards Billy. The table also includes the votes of the two groups who could not find an agreement, as they are still relevant within this case.

	Condition	Billy	Eddie	Mickey	Not valid
Questionnaire	Control	8	15	5	2
Final Votes	Control	9	19	2	0
Questionnaire	Experimental	8	13	8	1
Final Votes	Experimental	8	20	2	0

Table 4: Table reporting how participants voted for each of the three suspects.

Given the data, we wanted to correlate the groups that correctly voted Eddie with their questionnaire answers. We found that, among these 5 correct groups and their 15 participants, 7 participants appointed Eddie in the questionnaire, 5 appointed Billy, and 3 appointed Mickey. This was important to note as we found no correlation between the initial opinion of users that belonged to the correct groups and the result ($p=0.18$), meaning the groups that correctly guessed didn't do so simply because all three users had the same idea.

This being said, by reading the transcripts we argue that the issues with the results were strongly tied to how willing the users were to discuss the case, out of the 10 groups in the control condition only 4 passed the 10-minute mark, and only 2 have a thorough discussion that lasted more than 20 minutes. Groups like 6 and 7 are the causes for the low success rate, within these after the three users entered, and after establishing the prints on the murder weapon were Billy's, they all voted immediately without further discussion. On the other hand, pilot Groups and online groups 4 and 3, went further in the discussion, even if the fingerprints came up during the discussion participants wanted to fully explore other possibilities. This is well represented by the discussion time, which averages 13 minutes in the no moderator condition online, against the full 30 minutes of the pilots. It is important to remember that the original study's participants were all psychology students performing the study as a mandatory task, which likely ensured longer and more thorough conversations.

5.4. Experimental Condition (With the moderator)

Despite the results obtained in the control condition, we still expected some effects from the groups with the *AI Moderator*. Unfortunately, the moderator did not seem to lead to a higher success rate, as only 2 groups out of the ten in the condition correctly guessed the culprit. We want to note how Groups 16 and 14 failed to reach a consensus within the allotted time, and their votes were thus gathered separately.

Group ID	11	12	13	14	15	16	17	18	19	20	Average
Voted	Eddie	Billy	Billy	2 M 1 E	Eddie	2 B 1 E	Billy	Billy	Billy	Billy	Billy
Non-critical	8	10	18	22	22	14	9	11	5	8	12.7
Critical	1	1	2	2	5	2	0	2	0	1	1.6
Total info	9	11	20	24	27	16	9	13	5	9	14.3
Discussion Focus	11.11%	9.09%	10%	8.33%	18.51%	12.50%	0.0	15.38%	0.0	11.11%	9.60%
Info request	7	9	5	12	26	18	5	8	3	7	10
Info request/Total	12.50%	20%	7.35%	15.38%	24.07%	22.78%	12.19%	15.68%	17.64%	19.44%	16.70%
Inference	11	5	8	6	23	4	4	4	0	3	6.8
Inference/Total	19.64%	11.11%	11.76%	7.69%	21.29%	5.06%	9.75%	7.84%	0.0	8.33%	10.25%
Mod Messages	19	9	19	30	75	31	5	26	2	4	22
User Messages	56	45	68	78	108	79	41	51	17	36	57.90
Total Messages	75	54	87	108	183	110	46	77	19	40	79.9
Intervention rate	0.33	0.20	0.28	0.38	0.69	0.39	0.12	0.50	0.11	0.11	0.31
Discussion Time	10.13	10.65	17.31	30.36	30.14	30.52	12.73	22.61	9.34	12.67	18.65%
Gini Adjusted	0.05	0.22	0.12	0.20	0.09	0.16	0.21	0.10	0.17	0.13	0.15

Table 5: Table with the results obtained from groups in the moderator condition

Firstly, we looked to compare the total amount of information shared in each condition but found no statistical significance ($U=34.5$, $p=0.2546$). We also compared *Discussion focus*, to see if the moderator could lead to a richer discussion, but we found no significance in that either ($U=36.5$, $p=0.3149$). We also looked at the discussion times to see if a Whitney U test could highlight an increase in the experimental condition, but due to the high variance (no moderator $M=14.43$, $SD=10.05$, with moderator $M=18.65$, $SD=8.95$), no evidence was found ($U=29.0$, $p=0.1212$).

To further examine what effects the *AI Moderator* had in the chat, we examined the kinds of messages that were sent. We hypothesized that a moderator would lead the users toward a deeper understanding of the information, thus provoking more questions for unknown information. However, by analyzing the data, no significance was found between the number of messages containing Info Requests with and without the moderator ($U=34.0$, $p=0.2368$). We also tried scaling this information with the number of messages sent by users, as seen in the field *Inference/Total* in Table 5, but no significance was found ($U=41.0$, $p=0.5202$).

Similarly, we believed that a moderator could facilitate inferences by encouraging theory development or the emergence of new information. We looked at the number of Inferences, considering also their scaled version with

the number of messages sent by users. We looked if the *AI Moderator* caused a surge in either of these two, but unfortunately, no statistical evidence was found for either (Inference: $U=55$, $p=0.7329$, Inference/Total: $U=61$, $p=0.4273$).

After reviewing the data, we hypothesized that the moderator was being ignored, as no data showed statistical differences. To test this hypothesis, three researchers reviewed chats numbered 11 through 15 and attempted to determine whether the messages users were sending were related to what the moderator had said. Each researcher flagged each message with a 1 if it was related to the moderator’s input, and 0 otherwise. This analysis needed to be conducted by different individuals, as it involved interpreting participants’ thoughts while writing, making it a deeply subjective task. After completing their individual reviews, the researchers met to reach a consensus on which messages were related to the moderator’s input. The number of messages identified in this manner is reported in Table 6. As seen in the table, on average, about one-third of the messages were considered to be related to the moderator, indicating that the moderator was far from being ignored.

Group ID	11	12	13	14	15	Average
Responded	5	4	7	9	22	9.4
Mod Messages	19	9	19	30	75	30.4
Percentage	26.31%	44.44%	36.84%	30%	29.33%	33%

Table 6: Table highlighting how many messages from the moderator were considered.

As mentioned in Section 4.2, during development, we observed a tendency for the moderator to intervene more frequently as the chat progressed. We initially hypothesized that such behavior might cause participants to ignore the moderator. However, as previously explained, the final moderator was acknowledged by participants. To investigate whether this issue persisted, we introduced the *Intervention Rate*, as a measure of how often the *AI Moderator* replied in a chat (as explained in Section 5.2). Across all groups, the I_r averaged approximately 30%, indicating that, on average, one out of three user messages triggered a moderator response. We expected the I_r to be constant or, at most, random. However, we found a strong statistical correlation ($r=0.90$, $p<0.001$) between the *Intervention Rate* and the total number of messages, confirming that the *AI Moderator* increases its intervention frequency as the chat continues, with the longest chat having up to $I_r=80\%$. That being said, a higher frequency of intervention is not necessarily detrimental, as there is no evidence in our results to suggest that a lower frequency leads to better overall moderation.

To explore more about how this influenced *AI Moderator*, we analyzed the reviews participants left in the *Feedback Page* about the chatbot, to relate their experience with this effect. Of the 30 participants in the experimental condition, 22 completed the feedback question that read «*Considering the moderator chatbot: what worked well for you? What didn’t?*». We then gathered the replies and performed sentiment analysis on them using NLTK, a Python library fitted for the task. The results were normalized between 0 and 1, where 0 was a terrible experience and 1 was flawless. The average sentiment people displayed towards the moderator was 0.56, meaning the moderator was mostly perceived as ok. However, we wanted to see if a higher intervention rate resulted in a worse perceived moderator, so we correlated the intervention rate with the user’s perceived sentiments. The Pearson correlation between the two variables showed no statistical evidence ($r=-0.09$, $p=0.7035$), meaning that a very vocal moderator isn’t necessarily badly perceived.

We sometimes observed polar opposite reviews, even within the same group. For instance, in group 12, one user commented, «*He was behind a bit and a bit pushy.*», while another remarked, «*It was good to have someone direct our focus.*». This introduces an important point: not only does the moderator’s behavior vary from group to group, but the participants’ responses to that behavior also differ. This variability makes it challenging to find a solution that works well in all cases.

Since the murder mystery setting was also employed in a study on discussion balance [4], we wanted to explore whether a similar balance persisted in our data. To do so we analyzed the *Gini Index*, as a metric for balanced participation. We investigated if the inclusion of the *AI Moderator* could change the discussion balance, however, we found no statistical significance between the two conditions ($U=65.0$, $p=0.2730$). We further investigated whether a good balance could correlate to a correct outcome of the task, however, no correlation was found either ($\rho_b=-0.01$, $p=0.94$).

Given the nature of the *Hidden Profile* task, we explored whether a lower *Gini index*, signifying a more balanced

conversation, correlated with the number of critical clues shared. Again, we found no statistical evidence ($r=-0.1328$, $p=0.5767$). Additionally, we correlated the *Gini index* with the total amount of information shared and found no statistical evidence ($r=0.1197$, $p=0.6151$).

This result may be attributed to the nature of short group discussions, where all participants contributed only a few messages each (balanced conversation, low G_n^{Adj}) and shared few clues (low number of critical clues and overall information).

6. Limitations

The primary limitation of this study was the quality of the participants. Due to the requirement of having three individuals operate simultaneously, we opted to use Prolific rather than coordinating physical participants, which would have been too time-consuming. However, upon reviewing the results, we found that the participant quality was lower than anticipated, which significantly hindered the integrity of the study's data.

Furthermore, after reviewing the feedback and chats, we noticed some complaints about the bot being misleading. Particularly in group 18, the LLM began suggesting that participants wait for CCTV footage before choosing the culprit—something they obviously could not do. Similarly, during group 16's discussion, the moderator suggested that users reschedule the decision for a later date after reviewing additional information. The participants, unaware that the LLM could suggest options that were not feasible, spent time discussing which tool and date to use for a rescheduled discussion.

These two groups strongly highlighted the problem of a task-agnostic moderator. In both cases, not only was the AI unhelpful, but it was actively detrimental to achieving the goal. We must also clarify that these were not technically hallucinations. The LLM was not given any context within the prompt regarding the nature of the discussion, so it had no explicit rule to "break". Specifically, the prompt did not specify that further information could not be obtained, so the LLM did not violate any rules when it suggested rescheduling the decision for a later date. For future studies and efforts, an explanation of the operating conditions is needed.

From the reviews, we also noticed that in some groups, people perceived that the *AI Moderator* repeated itself. As previously discussed in Section 4.2, this issue was also observed during testing, but it appeared to be resolved after changing the API call. However, this problem is difficult to quantify, as what constitutes a "repetition" is not clearly defined. The bot does not repeat itself verbatim but may reiterate certain points across different messages. While the exact cause remains unclear, we hypothesized that the more context the moderator has to process, the more likely it is to reference the same points multiple times.

The same issue was addressed using summaries, as detailed in 4.2.4. However, our testing did not provide conclusive evidence that this approach was effective. We believe that with proper tuning, this method could potentially resolve the problem. However, this would involve adjusting the frequency of summaries (the *index* value) and refining the prompt used for summarization, or possibly employing a suitably fine-tuned BERT model.

Lastly, we want to address the issue of the content filter imposed by the Azure API. After reviewing the chat logs, we noted that only one or two messages were typically blocked per chat. While this number isn't particularly high, the problem lies in the kind of messages that are flagged. Because of the task's nature, the blocked messages often contain substantial amounts of information.

For example, in one group, the moderator asked participants for a summary of all the data they had gathered. One participant complied by submitting a long message filled with both critical and non-critical information. Unfortunately, since this information was linked to the murder weapon, the filter detected it as violent content, and the moderator ignored the message. Shortly after, another participant submitted a shorter message with information about the case, which was not flagged. This resulted in a strong bias in the moderator's response, as it only considered the second participant's input, ignoring the crucial details provided by the first participant.

7. Conclusion and Future Work

This Thesis investigated the role of a multi-user conversational AI as a moderator in group discussions. The study adapted a well-established murder mystery task for an online format, comparing conversations facilitated by an LLM moderator, powered by GPT-3.5, with those conducted without AI intervention.

The findings revealed that while the moderator influenced the structure of the conversation, its impact on decision-making accuracy was minimal. Contrary to expectations that the moderator would enhance informa-

tion sharing and improve focus, no significant differences were observed in critical metrics such as information sharing, discussion time, or message content between the moderated and non-moderated groups.

We partly attribute this outcome to the shift from a physical to a digital environment, which may have reduced engagement levels. Many participants appeared to rush through the discussion, likely contributing to discrepancies between the control condition and the results of the original study. This trend may have been exacerbated by the use of participants recruited from Prolific, a platform known for attracting users who prioritize speed over thoroughness in completing tasks [27].

Additionally, while the moderator generally intervened appropriately, the lack of participant engagement limited its ability to deepen the conversations effectively. The reviews indicated that different participants perceived the same moderator in various ways, with a more vocal moderator not being poorly received by most. This highlights the significant variance in multi-user studies—not only do groups differ from each other, but variability also exists within individual groups.

The Thesis also highlighted that the design of the *AI Moderator*, though functional, may have lacked the sophistication necessary to balance interventions effectively in real-time group discussions. At times, the *AI Moderator*’s lack of contextual understanding misled participants, and behaviors such as repeating the same points underscored the challenges inherent in developing task-specific AI for open-ended decision-making scenarios.

For our future works we plan to further explore this task, applying the lessons learned from this development. Our first step will be to replicate the control condition, this time using recruited university students instead of Prolific participants. This change is expected to enhance engagement and improve the quality of discussions, as students are likely to invest more effort in academic research settings. Additionally, the study format will be adjusted to ensure that each group fully utilizes the 30-minute discussion period. By encouraging deeper participant engagement, we aim to collect data comparable to the baseline study, as indicated by the findings from our pilot studies.

This Thesis was important in exploring the possibilities of a baseline prompt and highlighting its shortcomings. To address these issues, we first propose shifting from a task-agnostic prompt to a more specific one. During the study, it was observed that the lack of context sometimes led groups astray, undermining entire session outcomes. We hypothesize that providing full context at the beginning of the prompt would mitigate this issue. Below, we present an example of such a prompt:

As a highly intelligent AI system, you are the moderator (M) in a conversation between three participants (A, B, and C), who need to solve a murder mystery game. The three participants have reviewed documents regarding the mystery story for 30 minutes and are now discussing who the culprit may be based on this information...

Furthermore, as we shift away from a generalized solution, we will be free to explore more complex approaches. Consequently, adjustments will be made to the implementation of summaries in the prompt, as explained in Section 4.2.4. This approach has shown promise, and we hope that, if tuned, it will address issues related to the moderator repeating concepts.

Lastly, to reduce moderator bias within chats by removing filters, we will either aim to unlock Azure filters or transition to a new LLM entirely. This approach will also enhance the user experience by eliminating the need to check whether their message has been taken into account or not.

References

- [1] Md. Al-Amin, Mohammad Shazed Ali, Abdus Salam, Arif Khan, Ashraf Ali, Ahsan Ullah, Md Nur Alam, and Shamsul Kabir Chowdhury. History of generative artificial intelligence (ai) chatbots: past, present, and future development. *ArXiv*, abs/2402.05122, 2024.
- [2] Anthropic. Claude: A next-generation ai assistant, 2023.
- [3] Simran Arora, Avanika Narayan, Mayee F. Chen, Laurel Orr, Neel Guha, Kush Bhatia, Ines Chami, Frederic Sala, and Christopher Ré. Ask me anything: A simple strategy for prompting language models, 2022.
- [4] Khaled Bachour, Frdric Kaplan, and Pierre Dillenbourg. An interactive table for supporting participation balance in face-to-face collaborative learning. *IEEE Transactions on Learning Technologies*, 3:203–213, 2010.
- [5] Som Biswas. Prompt engineering for biomedical engineering. *Int J Clin Med Edu Res*, 2(7):194, 2023.
- [6] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020.
- [7] Rui Cao, Roy Ka-Wei Lee, Wen-Haw Chong, and Jing Jiang. Prompting for multimodal hateful meme classification, 2023.
- [8] Patrick Chao, Alexander Robey, Edgar Dobriban, Hamed Hassani, George J. Pappas, and Eric Wong. Jailbreaking black box large language models in twenty queries, 2024.
- [9] Banghao Chen, Zhaofeng Zhang, Nicolas Langrené, and Shengxin Zhu. Unleashing the potential of prompt engineering in large language models: a comprehensive review, 2024.
- [10] Maximillian Chen, Alexandros Papangelis, Chenyang Tao, Seokhwan Kim, Andy Rosenbaum, Yang Liu, Zhou Yu, and Dilek Hakkani-Tur. Places: Prompting language models for social conversation synthesis, 2023.
- [11] Chun Wei Chiang, Zhuoran Lu, Zhuoyan Li, and Ming Yin. Enhancing ai-assisted group decision making through llm-powered devil’s advocate. In *ACM International Conference Proceeding Series*, pages 103–119. Association for Computing Machinery, 3 2024.
- [12] Justin Cranshaw, Rafal Kocielnik, Bowen Yu, and Jaime Teevan. Calendar.help: Designing a workflow-based scheduling agent with humans in the loop. *SSRN Electronic Journal*, 01 2017.
- [13] George Deltas. The small-sample bias of the gini coefficient: results and implications for empirical research. *Review of economics and statistics*, 85(1):226–234, 2003.
- [14] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.
- [15] Tao Feng, Chuanyang Jin, Jingyu Liu, Kunlun Zhu, Haoqin Tu, Zirui Cheng, Guanyu Lin, and Jiaxuan You. How far are we from agi, 2024.
- [16] Mathyas Giudici, Giulio Antonio Abbo, Ottavia Belotti, Alessio Braccini, Francesco Dubini, Riccardo Andrea Izzo, Pietro Crovari, and Franca Garzotto. Assessing llms responses in the field of domestic sustainability: An exploratory study. In *2023 Third International Conference on Digital Data Processing (DDP)*, pages 42–48. IEEE, 11 2023.
- [17] Marie Goetz, Kathrin Wolter, and Michael Prilla. The influence of unequal chatbot treatment on users in group chat. In *Proceedings of Mensch Und Computer 2022*, MuC ’22, page 297–307, New York, NY, USA, 2022. Association for Computing Machinery.
- [18] Google. Gemini, 2024.

- [19] Ya-Hui Hsieh and Chin-Chung Tsai. The effect of moderator’s facilitative strategies on online synchronous discussions. *Computers in Human Behavior*, 28:1708–1716, 9 2012.
- [20] Shafquat Hussain, Omid Ameri Sianaki, and Nedat Ababneh. A survey on conversational agents/chatbots classification and design techniques. In *Web, Artificial Intelligence and Network Applications: Proceedings of the Workshops of the 33rd International Conference on Advanced Information Networking and Applications (WAINA-2019) 33*, pages 946–956. Springer, 2019.
- [21] Salman Khan, Muzammal Naseer, Munawar Hayat, Syed Waqas Zamir, Fahad Shahbaz Khan, and Mubarak Shah. Transformers in vision: A survey. *ACM Comput. Surv.*, 54(10s), sep 2022.
- [22] Soomin Kim, Jinsu Eun, Joseph Seering, and Joonhwan Lee. Moderator chatbot for deliberative discussion: Effects of discussion structure and discussant facilitation. *Proc. ACM Hum.-Comput. Interact.*, 5(CSCW1), apr 2021.
- [23] Richard A Krueger. *Moderating focus groups*. Sage, 1998.
- [24] Zine Eddine Latioui, Lamine Bougueroua, and Alain Moretto. *Social Media Chatbot System - Beekeeping Case Study*, pages 302–310. 2020.
- [25] Zi Liang, Haibo Hu, Qingqing Ye, Yaxin Xiao, and Haoyang Li. Why are my prompts leaked? unraveling prompt extraction threats in customized large language models, 2024.
- [26] Manqing Mao, Paishun Ting, Yijian Xiang, Mingyang Xu, Julia Chen, and Jianzhe Lin. Multi-user chat assistant (muca): a framework using llms to facilitate group conversations, 2024.
- [27] Eyal Peer, David Rothschild, Andrew Gordon, Zak Evernden, and Ekaterina Damer. Data quality of platforms and panels for online behavioral research. *Behavior Research Methods*, 54:1643–1662, 9 2021.
- [28] Amy K. Peterson and Cary J. Roseth. Effects of four cscl strategies for enhancing online discussion forums: Social interdependence, summarizing, scripts, and synchronicity. *International Journal of Educational Research*, 76:147–161, 2016.
- [29] Python Software Foundation. *asyncio — Asynchronous I/O*, n.d.
- [30] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving language understanding by generative pre-training. 2018.
- [31] Navin C. Sabharwal and Amit Agrawal. Introduction to google dialogflow. 2020.
- [32] Donghoon Shin, Soomin Kim, Ruoxi Shang, Joonhwan Lee, and Gary Hsieh. Introbot: Exploring the use of chatbot-assisted familiarization in online collaborative groups. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, CHI ’23, New York, NY, USA, 2023. Association for Computing Machinery.
- [33] Garold Stasser. Information salience and the discovery of hidden profiles by decision-making groups: A “thought experiment”. *Organizational Behavior and Human Decision Processes*, 52:156–181, 6 1992.
- [34] Garold Stasser and Dennis Stewart. Discovery of hidden profiles by decision-making groups: Solving a problem versus making a judgment. *Journal of personality and social psychology*, 63(3):426, 1992.
- [35] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023.
- [36] Dietrich Trautmann, Alina Petrova, and Frank Schilder. Legal prompt engineering for multilingual legal judgement prediction, 2022.
- [37] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.
- [38] A. Vinciarelli, A. Dielmann, S. Favre, and H. Salamin. Canal9: A database of political debates for analysis of social interactions. In *2009 3rd International Conference on Affective Computing and Intelligent Interaction and Workshops*, pages 1–4, 2009.

- [39] Nicolas Wagner, Matthias Kraus, Tibor Tonn, and Wolfgang Minker. Comparing moderation strategies in group chats with multi-user chatbots. *ACM International Conference Proceeding Series*, 7 2022.
- [40] Joseph Weizenbaum. Eliza—a computer program for the study of natural language communication between man and machine. *Communications of the ACM*, 9:36–45, 1 1966.
- [41] Yuxin Wen, Neel Jain, John Kirchenbauer, Micah Goldblum, Jonas Geiping, and Tom Goldstein. Hard prompts made easy: Gradient-based discrete optimization for prompt tuning and discovery, 2023.
- [42] Anbang Xu, Zhe Liu, Yufan Guo, Vibha Sinha, and Rama Akkiraju. A new chatbot for customer service on social media. CHI '17, New York, NY, USA, 2017. Association for Computing Machinery.
- [43] Qingxiao Zheng, Yiliu Tang, Yiren Liu, Weizi Liu, and Yun Huang. Ux research on conversational human-ai interaction: A literature review of the acm digital library. *Conference on Human Factors in Computing Systems - Proceedings*, 4 2022.

Abstract in lingua italiana

L'integrazione di Large Language Model (LLM) come GPT-3.5 in conversazioni multiutente ha il potenziale di influenzare le dinamiche sociali e i processi decisionali. Questa tesi esplora il ruolo di un chatbot alimentato dall'Intelligenza Artificiale (IA) come moderatore nelle discussioni di gruppo, verificando se può replicare un moderatore umano guidando le conversazioni verso risultati più efficaci. La ricerca si avvale di uno studio online in cui i partecipanti si cimentano in un "murder mystery", progettato per provocare la collaborazione e il processo decisionale, in due condizioni: con e senza un moderatore IA.

I nostri risultati indicano che, sebbene la moderazione dell'IA possa influenzare le dinamiche di partecipazione e focalizzare le discussioni, il suo impatto complessivo sull'accuratezza delle decisioni è stato limitato. I pilot hanno suggerito che alcuni problemi possono essere attribuiti alle caratteristiche del gruppo di partecipanti, che potrebbero aver influenzato i risultati. Ciononostante, riportiamo in dettaglio tutte le analisi, le fasi di progettazione e gli adattamenti per supportare ulteriori ricerche in questo campo. Queste intuizioni forniscono una solida base per il lavoro futuro sul miglioramento della moderazione guidata dall'intelligenza artificiale in contesti collaborativi.

Parole chiave: Interazione Uomo-Macchina, Interazione Uomo-IA, IA Generativa, Moderazione, Chatbot Multi-utente

Acknowledgements

Ringrazio dal più profondo del cuore tutte le persone che mi hanno accompagnato in questo percorso universitario.

Voglio prima di tutto ringraziare i professori che mi hanno seguito: ringrazio la Prof. Franca Garzotto per avermi dato fiducia ed avermi proposto questo progetto, e ringrazio il Prof. Enrico Costanza per l'accoglienza dimostratami durante i miei 3 mesi a Londra.

Ringrazio infinitamente il mio tutor, Mathyas, che da amico mi ha seguito per la buona parte di un anno.

Ringrazio mia mamma, mio papà, mio fratello e la mia famiglia per avermi sostenuto ed aver creduto in me in questi anni universitari. Grazie.

Ringrazio inoltre i miei amici di una vita: Riccardo, Vittorio, Matteo, Andrea, Matteo, Alessandro, Riccardo, Roberto e Federico. Mi avete negli anni aiutato a crescere nella persona che sono ora.

Un pensiero inoltre ai miei compagni in armi: Marcello, Marcello, Emanuele, Martin, Arya, Alessio, Riccardo e Ottavia. Siete stati grandi fonte di spunto e confronto, e grazie alla vostra presenza in aula molte lezioni sono passate in modo più spensierato.

Ringrazio tutte le altre persone che mi hanno aiutato e che per motivi di spazio non posso elencare.

Infine pongo un particolare ringraziamento a Teresa, perché con la sua forza ha saputo aiutarmi e sostenermi nei momenti più difficili.